Public Key Encryption from High-Corruption Constraint Satisfaction Problems

Isaac M Hair [*] UCSB, UCLA

Amit Sahai [†] UCLA

April 12, 2026

Abstract

We give a public key encryption scheme with plausible quasi-exponential security based on the conjectured intractability of two constraint satisfaction problems (CSPs), both of which are instantiated with a corruption rate of $1 - o(1)$. First, we conjecture the hardness of a new large alphabet random predicate CSP (LARP-CSP) defined over an arbitrary but strongly expanding factor graph, where the vast majority of predicate outputs are replaced with random outputs. Second, we conjecture the hardness of the standard kXOR problem defined over a random factor graph, again where the vast majority of parity computations are replaced with random bits. In support of our hardness conjecture for LARP-CSPs, we give a variety of lower bounds, ruling out many natural attacks including all known attacks that exploit non-random factor graphs.

Our public key encryption scheme is the first to leverage high corruption CSPs while simultaneously achieving a plausible security level far above quasi-polynomial. At the heart of our work is a new method for planting cryptographic trapdoors based on the *label extended factor graph* for a CSP.

Along the way to achieving our result, we give the first uniform construction of an error-correcting code that has an expanding, low density generator matrix while simultaneously allowing for efficient decoding from a $1 - o(1)$ fraction of corruptions.

[*] isaacmhair@gmail.com

[†] sahai@cs.ucla.edu This research was supported in part from a Simons Investigator Award, DARPA SIEVE award, NTT Research, NSF grant 2333935, BSF grant 2022370, a Xerox Faculty Research Award, a Google Faculty Research Award, an Okawa Foundation Research Grant, and the Symantec Chair of Computer Science. This material is based upon work supported by the Defense Advanced Research Projects Agency through Award HR00112020024.

1 Introduction

Public key encryption (PKE) [GM84] is a fundamental cryptographic primitive that enables secure transmission of data between two parties over an insecure channel with no prior communication. Despite several decades of research, there are very few sources of computational hardness from which we know how to build PKE, mainly: number-theoretic problems [DH76, RSA78, Rab79], and coding-theoretic/lattice problems [McE78, Ale03, Reg09]. It's known that the number-theoretic problems are broken by large scale quantum computers [Sho99], which leaves the troublesome possibility that a mathematical breakthrough on the coding-theoretic/lattice problems could render almost all PKE schemes insecure. As such, in the words of Applebaum, Barak, and Wigderson [ABW10],

"A major goal of cryptography is to base public-key encryption on assumptions that are weaker, or at least different, than those currently used."

The major challenge in designing any public key encryption scheme is finding methods for planting "trapdoors" that allow for decryption, while still ensuring hardness of breaking the encryption. Despite decades of research, we have few techniques. Developing new techniques, which is the focus of this work, spurs research in both the cryptographic and algorithms communities. The value of research in this direction is multifaceted. First and foremost, finding a broader set of hardness conjectures that imply PKE gives us more confidence in the existence of this primitive, even in a post-quantum world. On a more fundamental level, this type of research gives us a better understanding of the mathematical nature of PKE, and the relationship between PKE and average-case hardness of NP problems.

Adopting a high-error CSP Perspective on PKE. In this paper we build a public key encryption scheme from a new source of hardness: *very high corruption* constraint satisfaction problems (CSPs).[1] Constraint satisfaction problems are a foundational topic in average-case complexity (see [KZ17] for a survey), and we have a plethora of lower bounds [BSI99, IPS99, Gri01, Sch08, Nor14, FPV15, AL16, AL16, MW16, KMOW17, CMMV17, WEAM19, Zhu20, MN24] that give a well defined region in which CSPs appear to be computationally intractable.

The CSPs of interest to us will take the following form. We are given a set of n variables and a set of $m = \text{quasipoly}(n)$ constraints, each of which depends on a size $k = \text{polylog}(n)$ subset of the variables. The task is to determine whether there is a hidden "planted" assignment to the variables which satisfies an unusually large number of the constraints, or whether the constraints do not respect any planted assignment. Our CSPs will have a *corruption rate* of $1-o(1)$, meaning that (in the case there does exist a planted solution) each constraint is replaced with a corrupted version of itself independently with probability $1 - o(1)$. The security of our PKE scheme will follow from the intractability of two specific CSPs in this high corruption regime:

1. A new *large alphabet, random predicate* CSP (LARP-CSP), where the factor graph for constraints and variables is arbitrary but required to be strongly expanding. LARP-CSPs are closely related to Goldreich's pseudorandom generator [Gol00] and planted hypergraph problems [DMW25]. We give several lower bounds, ruling out many natural attacks on LARP-CSPs *including all known attacks which exploit structured factor graphs* [OST18]. Intuitively, the conjectured hardness of LARP-CSPs comes from the massive entropy contributed by the random predicates, and LARP-CSPs appear to be computationally intractable even in the *corruption-free* regime. (But for our PKE scheme all LARP-CSPs will have a $1 - o(1)$ corruption rate.)

[1] Our work has led to follow-up work that also uses our perspective [Man26].

2. The standard kXOR problem, where the factor graph for constraints and variables is chosen at random. The main difference between our kXOR problem and more typical versions used for cryptogrpahy is that our corruption rate will be $1 - o(1)$, as opposed to inverse polynomial.

A New Way to Plant Trapdoors. In this paper we introduce the idea of using the *label extended factor graph* for one CSP to plant a trapdoor in another CSP. Arguments based on label extended factor graphs are common in average-case complexity [Ste10, GS11, BCG+12, AL17, CMMV17], but our work is the first to leverage them for buildin cryptographic trapdoors[2].

Along the way to designing our trapdoor technique and decryption algorithm, we give the first *uniform* construction of an error correcting code with an expanding, low density generator matrix that allows for efficient decoding from a $1 - o(1)$ fraction of corruptions. A previous work by Oliveira, Santhanam, and Tell [OST18] achieved this goal only non-constructively: they proved the *existence* of expander graphs that could be interpreted as giving rise to such a code, but did not show how to construct it.

Beyond Quasi-Polynomial Security. There are a couple existing PKE schemes [ABW10, YZ16] that can be re-framed as relying on the conjectured intractability of high-corruption constraint satisfaction problems. However, all of these schemes achieve a security level of at best $\lambda^{O(\log \lambda)}$, which is quasi-polynomial. The issue comes from a **brute-force decryption barrier.** One way to interpret decryption in these schemes is as executing an exponential-time guess-and-check algorithm on a logarithmic size hidden component in the ciphertext. Because the size of the component itself is only logarithmic, an adversary can always guess its location in time $\lambda^{O(\log \lambda)}$.

Our PKE scheme is the first to leverage high-corruption CSPs without resorting to a brute-force-type decryption algorithm, enabling us to achieve a significantly higher security level. In particular, our public key encryption scheme achieves a plausible *quasi-exponential* (see [Sah26]) security level of $\exp\left\{2^{\log^{\Omega(1)} \lambda}\right\}$, where $\lambda^{O(1)}$ is the running time of the algorithms in our scheme. This is significantly larger[3] than quasi-polynomial $\lambda^{O(\log \lambda)}$ and comes close to sub-exponential security, which would be of the form $\exp\{\lambda^c\}$ for a constant $c \in (0, 1)$.

On the "Win-Win" Perspective for Hardness Conjectures for Cryptography. One of the great advances in cryptography that occurred with the advent of probabilistic encryption [GM84] and related breakthroughs is what is known as the "*win-win*" perspective with regard to hardness assumptions (see, e.g., [GTK15] and references therein). This posits that, ideally, hardness conjectures that are useful for cryptography should be longstanding conjectures in mathematics, like the hardness of integer factorization. When this is so, then either we get a secure cryptosystem, or we refute a longstanding conjecture! This is a beautiful perspective, especially from the point of view of designing cryptosystems with an eye toward real-world deployment.

But what about theoretical investigations into foundations of cryptography, like the present research? Indeed, if we look historically, we find that many longstanding mathematical conjectures central to cryptography actually *arose* from cryptography, perhaps most notably the Diffie Hellman problem, where the Diffie Hellman problem was based on the longstanding conjecture that Discrete Logarithm is hard. Similarly, the Decisional Unbalanced Expansion (DUE) Assumption from [ABW10] had not been posed earlier, and was far from a longstanding conjecture in mathematics, but it too was inspired by the longstanding conjecture that the Densest Subgraph Problem is hard on average.

[2]Our work has already led to follow-up work [Man26] that makes use of the label extended factor graph for planting trapdoors in the context of a different CSP.

[3]We refer the reader to [Sah26] for a discussion of why quasi-exponential security is a natural notion, and why it is more similar to subexponential security than to quasi-polynomial security.

Nevertheless, we argue that there is a strong "win-win" flavor to these hardness conjectures, including our new large-alphabet random-predicate CSP hardness conjecture, which itself is inspired by the longstanding conjecture that CSPs are hard to solve on average. Namely, by posing these hardness conjectures, that are proven to be useful cryptographically, we create a scientific environment where *breaking these conjectures would provide critically needed understanding* about what freedom we have in co-designing cryptographic schemes and the hardness conjectures they rely on.

Indeed, this is not speculation: the goal of building indistinguishability obfuscation schemes was achieved [JLS21] only through a nearly decade-long research community process of exploring many hardness conjectures that turned out to be broken (e.g. Annihilation Attacks [MSZ16] breaking the hardness conjecture underlying the first cryptographic iO scheme [GGH+16]). In our context, one win becomes "deepening our understanding of the hardness of natural average-case problems", while the other stays "having a secure cryptosystem." Indeed, recent evidence suggests how useful it can be to posit general recipes for hardness like the low degree conjecture of Hopkins [Hop18] even when the conjecture is subject to attacks [BHJK25].

We believe that adopting this wider "win-win" perspective, already implicitly advocated by [ABW10], is crucial to developing deeper and more diverse algorithmic foundations for cryptography.

1.1 Large Alphabet Random Predicate CSPs

In this subsection we discuss our large alphabet random predicate CSPs, and present several lower bounds. We start with a few basic definitions (see Section 2 for details). An (m, n, k)-matrix is a matrix $\mathbf{H} \in \mathbb{F}_2^{m \times n}$ such that each row has exactly k nonzero entries. We say that such a matrix is a (γ, t)-expander if for every row-induced submatrix $\mathbf{H}'$ containing t' rows with $1 \leq t' \leq t$, there are at least $\gamma k t'$ distinct columns containing a nonzero entry of $\mathbf{H}'$. For values of γ close to 1, this means that every subset of at most t rows has nearly all of its nonzero entries lying in distinct columns, i.e. we have strong boundary expansion. We use $N_{\mathbf{H}}(i, j)$ to denote the *column index* of the jth nonzero entry in the ith row of $\mathbf{H}$.

Below we give our hardness conjecture for large alphabet random predicate CSPs. See Section 3 Conjecture 3.1 for a slightly more general form.

Conjecture 1.1 (High Corruption LARP-CSP Conjecture)**.** *Let $\mathbf{H}$ be any $(1 - o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix,*[4] *where $k = (\log n)^{\Theta(1)}$ and $m \leq n^{o(k)}$, and let $\alpha(n)$ be any function that grows as $1 - o(1)$. Let Σ, Γ be any alphabets satisfying $|\Sigma| = (nm)^{\log^{\Theta(1)}(nm)}$ and $|\Gamma| \leq |\Sigma|^{3k/4}$. Sample m random functions $f_i : \Sigma^k \to \Gamma$, and let $\mathcal{F}$ be the set of all f_i.*[5] *Then no $\mathrm{poly}(|\Sigma|^k)$ size algorithm can distinguish, with advantage more than $1/4$, between the following two distributions.*

1. *Null distribution: $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, where $\mathbf{b} \in \Gamma^m$ is sampled at random.*
2. *Planted distribution: $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, where $\mathbf{b} \in \Gamma^m$ is sampled by picking $\mathbf{s} \in \Sigma^n$ at random and setting*

$$\mathbf{b}_i = \begin{cases} \textit{Random element of } \Gamma, & \textit{with probability } \alpha = 1 - o(1). \\ f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)}), & \textit{otherwise.} \end{cases}$$

In other words, the conjecture states that for any *highly expanding* matrix $\mathbf{H}$, if we sample constraints defined by pairs $(f_i, \mathbf{b}_i)$ conditioned on obeying a planted assignment, but then corrupt each constraint with

[4] For our PKE scheme to be secure, we only need this conjecture to hold for a specific distribution over expanders. We state the conjecture in a more general form to encourage algorithmic research.

[5] Technically, our "predicate functions f_i" do not fit the definition of a predicate, because they have non-binary output. But they can be re-phrased in terms of a predicate f_i' by setting $f_i'(\sigma_1, \ldots, \sigma_k) = 1$ if and only if $f_i(\sigma_1, \ldots, \sigma_k) = \mathbf{b}_i$. The problem is now to determine if there exists an assignment $\mathbf{s} \in \Sigma^n$ such that approximately a $1 - \alpha$ fraction of the predicates evaluate to 1.

probability $1 - o(1)$, the result is indistinguishable from a set of random constraints, for distinguishers of size polynomial in the description of the random functions f_i. Note that the domain for each f_i will in fact be *super-polynomially* larger than the size of the factor graph. So the entropy contribution of a *single* totally random constraint $(f_i, \mathbf{b}_i)$ is super-polynomially larger than the entropy contribution that would have been present if the *entire* factor graph had been chosen at random.

Observe that Conjecture 1.1 is closely related to Goldreich's pseudorandom generator (Goldreich's PRG) [Gol00]. The main differences are that in Goldreich's PRG the seed $\mathbf{s}$ is a Boolean vector, and there is no corruption of the output vector $\mathbf{b}$. We show in Section 6.2 that Conjecture 1.1 can also be viewed in terms of a planted hypergraph problem, where the observed hypergraph is defined over the vertex set $[n] \times \Sigma$, and we add an edge of arity k to represent for each $i \in [m]$ the preimages of $\mathbf{b}_i$ for the function f_i.

There are a few papers which explicitly make use of hardness assumptions where the problem is defined over an arbitrary, non-random expanding matrix/hypergraph, albeit in the setting of polynomial stretch. To give a non-exhaustive summary,

1. Applebaum and Raykov ([AR16], Assumption 1) used the existence of local pseudorandom generators with arbitrary expanding factor graphs to get highly efficient constructions of pseudorandom functions. Their assumption remains unbroken in part because of their *choice of predicate*.

2. Ghosal, Hair, Jain, and Sahai ([GHJS25], Conjecture 1.3) used the assumed hardness of noisy kLIN over arbitrary expanding factor graphs to help build public key encryption from the planted clique conjecture. Their assumption sidesteps known attacks in part because each *coefficient* of each linear predicate is chosen at random, which injects some extrinsic entropy into the problem.

3. Elimelech and Huleihel [EH25] explore statistical-computational gaps for a variety of planted detection problems, with one of their contributions being a set of lower bounds for detecting arbitrary planted structures in random graphs.

Below we give lower bounds in support of the High Corruption LARP-CSP Conjecture. *All of our lower bounds will be for the corruption-free version of the problem in the conjecture.*

Low Complexity Embedding Attacks. Oliveira, Santahnam, and Tell showed that in the setting of quasi-polynomial stretch, Goldreich's PRG is broken when instantiated with a very carefully chosen expanding factor graph [OST18]. Could similar attacks impact Conjecture 1.1?

In Section 6.1, we define the notion of a *low complexity embedding attack*, which captures and generalizes the Oliveira-Santhanam-Tell attacks. Then we show that *the entropy of the random functions alone is enough for the decision problem in Conjecture 1.1 to unconditionally resist attacks of this form, even in the errorless regime.* As far as are aware, ours is the first formalization of any lower bound against this family of attacks.

The Oliveira-Santhanam-Tell family of attacks works as follows. First exhibit a specific expanding matrix $\mathbf{H}$ and a specific predicate f such that both are described by a small $\mathrm{AC}^0[+]$ circuit. Then *no matter the choice of seed*, the output $\mathbf{b}$ of Goldreich's PRG when instantiated with $\mathbf{H}$ and f will be the truth table of a small $\mathrm{AC}^0[+]$ circuit. Using *natural property algorithms* (see e.g. [Raz87, Smo87, RR94, CIKK16]) we can distinguish such a vector $\mathbf{b}$ from a random vector, with at least constant advantage.

Another way to view the attacks is as follows. Let $\mathcal{L}$ be the set of all truth tables for small $\mathrm{AC}^0[+]$ circuits. The distinguisher works by simply measuring the Hamming distance from an observed vector $\mathbf{b}$ to the closest member of $\mathcal{L}$ and returning "planted" or "null" based on whether the distance is small or large, respectively. In the context of the LARP-CSP Conjecture, we generalize this attack in several ways, the most important of which are:

- We allow the adversary to pick any expanding matrix $\mathbf{H}$ *along with any subset* $\mathcal{L}$ against which to compute the distance, with the only requirement being that $\mathcal{L}$ is fixed before the functions in $\mathcal{F}$ are fixed (without this requirement every assumption would be broken).

- We allow the adversary to define *any coordinate-wise embeddings* that will be applied to an observed vector $\mathbf{b}$ before measuring its distance to the closest member of $\mathcal{L}$. Critically, the adversary is allowed to choose the coordinate-wise embeddings *after gaining knowledge* of the functions in $\mathcal{F}$.

In Theorem 6.2 we show that all attacks of this form will have distinguishing advantage at most $|\Sigma|^{-\Omega(k)}$, even for the corruption-free version of the LARP-CSP Conjecture, *and even when allowing the adversary unbounded running time to pick the best subset $\mathcal{L}$, to pick the best coordinate-wise embeddings, and to compute the distance to $\mathcal{L}$.*

Low Degree Polynomial Algorithms. Many efficient algorithms for planted detection problems can be represented in terms of a low degree polynomial over the reals. In fact, low degree polynomials give the best known algorithms for planted clique, sparse PCA, community detection, p-spin optimization, and many versions of the planted constraint satisfaction problem [KWB19].

The connection between low degree polynomials and efficient algorithms is not merely circumstantial. The best low degree polynomial algorithm for any hypothesis testing problem is the *low degree truncation* of a certain optimal hypothesis testing algorithm given by the Neyman-Pearson Lemma (see [KWB19] for a discussion). Recent papers [FGR+17, BBH+20] suggest a nearly tight correspondence between lower bounds against low degree polynomial algorithms and against *statistical query algorithms*. Under the Pseudocalibration Conjecture ([HKP+17] Conjecture 1.2), lower bounds against low degree polynomial algorithms for certain "well behaved" hypothesis testing problems imply lower bounds against *sum-of-squares algorithms* with comparable degree.

For these reasons, proving lower bounds against low degree polynomials is commonly used to support computational hardness conjectures for problems in average case complexity [BKW19, Kun21, BBK+21, Wei22, Wei23, DDL23, KVWX23, YZZ24, LG24, MW25] and cryptography [BKR23, BJRZ24].

In Section 6.2 Theorem 6.8, we rule out degree-$n^{0.99}$ polynomial algorithms for the problem in the LARP-CSP Conjecture, even in its corruption-free form. In many ways degree-$\leq n^{0.99}$ polynomials are a proxy for "combinatorial" algorithms that run in time $\exp\left\{n^{0.99}/\log^{O(1)} n\right\}$ [KWB19], so if degree-$\leq n^{0.99}$ polynomials are ineffective for the hypothesis testing problem this gives strong evidence that $\exp\left\{n^{0.99}/\log^{O(1)} n\right\}$ time "combinatorial" algorithms are also ineffective. Denoting by λ the running time of the algorithms in our PKE scheme, the lower bound becomes $\exp\left\{2^{\log^{\Omega(1)} \lambda}\right\}$.

Polynomial Calculus Refutations. The polynomial calculus proof system was introduced by Clegg, Edmonds, and Impagliazzo as one way to formalize linear-algebraic algorithms such as Gaussian elimination and basic Gröbner basis computations[6] [CEI96]. The ability to model these linear-algebraic algorithms makes polynomial calculus a very different algorithmic framework from e.g. low degree polynomial algorithms, sum-of-squares algorithms, and statistical query algorithms. Lower bounds against polynomial calculus refutations have been used to suggest hardness for various CSPs including Goldreich's PRG and random SAT instances [BSI99, IPS99, Nor14, AL16, MN24].

For the corruption-free version of the problem in the LARP-CSP Conjecture, the adversary is given a tuple $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, and the goal is to certify that there does *not exist* a secret vector $\mathbf{s} \in \Sigma^n$ such that $\mathbf{b}_i = f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)})$ for all $i \in [m]$, i.e. the system is *unsatisfiable*. The first step is to initialize a system of polynomials over a finite field (typically $\mathbb{F}_2$) that encode for all $i \in [m]$ the equation $\mathbf{b}_i =$

[6] In fact, Clegg, Edmonds, and Impagliazzo referred to the polynomial calculus proof system as the "Gröbner proof system."

$f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)})$. Then the adversary chooses any sequence of *derivations* that are permitted by the polynomial calculus proof system. The goal is to derive the equation $1 = 0$, which is only possible if the starting system was unsatisfiable. Using this approach we get a distinguisher with one-sided error, because the adversary can return "null distribution" if the refutation is successful and otherwise return "unsure." This type of one-sided error is present in other common frameworks such as the sum of squares hierarchy.

In Section 6.3 Theorem 6.13, we show that unsatisfiable $(\mathbf{H}, \mathcal{F}, \mathbf{b})$ tuples do not have polynomial calculus refutations over $\mathbb{F}_2$ of size less than $\exp\{n^{0.99}\}$, which translates into a size lower bound of $\exp\left\{2^{\log^{\Omega(1)} \lambda}\right\}$. *This holds even when the adversary is permitted unbounded time to choose the best embedding from Σ to a vector over $\mathbb{F}_2$ for each variable representing the vector* $\mathbf{s}$. Our proof techniques are inspired by a work of Applebaum and Lovett [AL16], in which they prove similar lower bounds for Goldreich's PRG.

1.2 Random kXOR

Below we give our hardness conjecture for random kXOR. A slightly more general form is stated in Section 3 as Conjecture 3.2.

Conjecture 1.2 (High Corruption Random kXOR Conjecture)**.** *Let $\mathbf{H}$ be a random (m, n, k)-matrix, where $k = (\log n)^{\Theta(1)}$ and $m \leq n^{k/3}$, and let $\beta(n)$ be any function that grows as $1 - o(1)$. Then no poly(m) size algorithm can distinguish, with advantage more than $1/4$, between the following two distributions.*

1. *Null distribution: $(\mathbf{H}, \mathbf{b})$, where $\mathbf{b} \in \mathbb{F}_2^m$ is sampled at random.*
2. *Planted distribution: $(\mathbf{H}, \mathbf{b})$, where $\mathbf{b} \in \mathbb{F}_2^m$ is sampled by picking $\mathbf{s} \in \mathbb{F}_2^n$ at random and setting*

$$\mathbf{b}_i = \begin{cases} \textit{Random element of } \mathbb{F}_2, & \textit{with probability } \beta = 1 - o(1). \\ \mathbf{s}_{N_{\mathbf{H}}(i,1)} + \ldots + \mathbf{s}_{N_{\mathbf{H}}(i,k)}, & \textit{otherwise.} \end{cases}$$

Constant-corruption kXOR is a classic problem in learning theory for all choices of m, e.g. $m = n^{O(1)}, m = n^{\log^{O(1)} n}$, and $m = \exp\left\{n^{O(1)}\right\}$ [FGKP06, Val12, CSZ24], and it is widely believed to be computationally intractable for $\exp\left\{n^{1-\Omega(1)}\right\}$ size adversaries. For perspective, the best known algorithms for the related constant-noise LPN problem (which is just kXOR but where the rows of $\mathbf{H}$ are dense random vectors) run in barely subexponential time: Blum, Kalai, and Wasserman gave an algorithm running in time $2^{O(n/\log n)}$ when $m = 2^{\Omega(n/\log n)}$ [BKW03], and Lyubashevsky gave an algorithm running in time $2^{O(n/\log\log n)}$ when $m = n^{1+\varepsilon}$ [Lyu05].

Lower Bounds for the High Corruption Random kXOR Conjecture. Applebaum, Barak, and Wigderson [ABW10] showed that whenever the matrix $\mathbf{H}$ is chosen at random and is of height $n^{(1/2-\Omega(1))k}$ (which is the case for our problem), the smallest subset of coordinates of $\mathbf{b}$ that have any bias towards zero will be of size $t = n^{\Omega(1)} = \exp\left\{2^{\log^{\Omega(1)} \lambda}\right\}$. This means that the problem in Conjecture 1.2 fools all local algorithms and linear tests. They also point out that we can apply a result of Braverman [Bra08] to show that any AC^0 circuit of size $\exp\left\{n^{o(1)}\right\}$ has distinguishing advantage $o(1)$ for the problem in Conjecture 1.2. This translates into a lower bound of $\exp\left\{2^{\log^{\Omega(1)} \lambda}\right\}$ on the size of any AC$^0[+]$ circuit with non-vanishing distinguishing advantage.

Grigoriev [Gri01], Schoenebeck [Sch08], and Kothari, Mori, O'Donnell, and Witmer [KMOW17] gave successively stronger/more general sum-of-squares lower bounds for CSPs, including random kXOR as an important special case. In our setting, these results show that any sum of squares algorithm for the problem

in Conjecture 1.2 with constant distinguishing advantage must have degree $n^{\Omega(1)}$. These algorithms run in $\exp\left\{n^{\Omega(1)}\right\} = \exp\left\{2^{\log^{\Omega(1)}\lambda}\right\}$ time.

Wein, El Alaoui, and Moore [WEAM19] gave a unified framework for analyzing belief propagation and approximate message passing algorithms from statistical physics, showing that these algorithms encounter the same barrier as sum-of-squares when applied to random kXOR. As such these results imply that any belief propagation/approximate message passing algorithm that has constant distinguishing advantage for the problem in Conjecture 1.2 must run in time $\exp\left\{2^{\log^{\Omega(1)}\lambda}\right\}$. Feldman, Perkins, and Vempala [FPV15] gave lower bounds for a powerful class of statistical query algorithms that also matches this lower bound.

1.3 Our Results and Techniques

Our main result is to construct a semantically secure public key encryption scheme from CSPs with a corruption rate of $1 - o(1)$. We prove the following theorem in Section 5.3.

Theorem 1.3. *Suppose that Conjecture 1.1 (High Corruption LARP-CSP) and Conjecture 1.2 (High Corruption kXOR) both hold. Then there is a semantically secure public key encryption scheme.*

At the heart of our result is a technique which *generically* allows us to construct a PKE scheme from a certain type of linear error correcting code that can handle a large fraction of erasures and corruptions. We now give an informal definition of the code; for a formal version see Definition 4.1. Note that all vectors are column vectors unless stated otherwise.

Definition 1.4 ((α, β)-Decodable Expanding Code (Informal))**.** *An (α, β)-Decodable Expanding Code is a tuple* $(\mathsf{MatrixGen}, \mathsf{Distinguish}, \alpha, \beta)$ *where* MatrixGen *and* Distinguish *are algorithms, α is an* erasure rate *parameter, and β is a* corruption rate *parameter. We require that*

- MatrixGen *outputs a generator matrix $\mathbf{G}$ for a linear code. We require that $\mathbf{G}$ is a $(1-o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix, where $k = \log^{\Theta(1)} n$ and $m = n^{o(k)}$.*
- *The algorithm* Distinguish[7] *can differentiate between*
  - *Random vectors in $\mathbb{F}_2^m$ where each entry is erased with probability α.*
  - *Noisy codewords $\mathbf{c}'$, where $\mathbf{c}'$ is sampled by taking a random codeword $\mathbf{c} \in \{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$, erasing each entry with probability α, and then corrupting (replacing with a random value) each remaining entry with probability β.*

We give an explicit, uniform, unconditional construction of an $(\alpha = 1 - o(1), \beta = 1 - o(1))$-decodable expanding code based on low-rate Reed-Muller codes, with our main contribution being an efficient algorithm to sample an expanding generator matrix; see Theorem 1.7 later in this subsection. In Section 4.4 we use such a code to build a PKE scheme (see Theorem 4.13).

Theorem 1.5 (Informal)**.** *Suppose we have an (α, β)-decodable expanding code. Then assuming Conjecture 1.1 (High Corruption LARP-CSP) holds with corruption rate α, and Conjecture 3.2 (High Corruption kXOR) holds with corruption rate β, there is a semantically secure PKE scheme.*

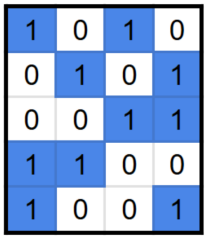
Notice that the *erasure rate* α from the (α, β)-decodable expanding code becomes the *corruption rate* for the LARP-CSP. The above theorem in fact holds for all choices of α and β, but for our scheme we always assume $\alpha = 1 - o(1)$ and $\beta = 1 - o(1)$.

[7] We only need a distinguisher, as opposed to a decoder. It's simple to convert from a decoder to a distinguisher, as we explain later in the paper.

To prove the theorem we give a direct construction of a PKE scheme that has a mild notion of correctness and security. From here, we can apply a black-box amplification theorem of Holenstein and Renner [HR05] to get semantically secure PKE. Below, we give an informal version of the starting PKE scheme; to ensure clarity, some important technical details are omitted.

Key Generation

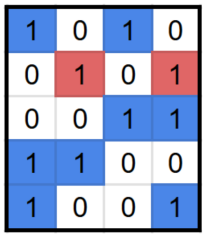
1. Sample a $(1-o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix $\mathbf{G}$ using the algorithm MatrixGen. Let Σ be a sufficiently large alphabet, and let Γ be another alphabet of size $|\Gamma| = |\Sigma|^{3k/4}$.



An example matrix $\mathbf{G}$ where $m = 5, n = 4$, and $k = 2$. For an actual instantiation of the PKE scheme, all these parameters would of course be significantly larger.

2. Sample random functions $f_1, \ldots, f_m$ and $\mathbf{b}$ from the planted distribution in the High Corruption LARP-CSP Conjecture. That is, we sample a set of random functions $\mathcal{F} = \{f_i : \Sigma^k \to \Gamma\}_{i \in [m]}$ and a secret $\mathbf{s} \in \Sigma^n$, then set

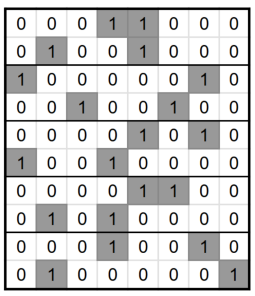
$$\mathbf{b}_i = \begin{cases} \text{Random element of } \Gamma, & \text{with probability } \alpha = 1 - o(1). \\ f_i(\mathbf{s}_{N_{\mathbf{G}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{G}}(i,k)}), & \text{otherwise.} \end{cases}$$



$\mathbf{b}_1 = f_1(\mathbf{s}_1, \mathbf{s}_3) = f_1(4, 5)$
$\mathbf{b}_2 =$ random
$\mathbf{b}_3 = f_3(\mathbf{s}_3, \mathbf{s}_4) = f_3(5, 7)$
$\mathbf{b}_4 = f_4(\mathbf{s}_1, \mathbf{s}_2) = f_4(4, 2)$
$\mathbf{b}_5 = f_5(\mathbf{s}_1, \mathbf{s}_4) = f_5(4, 7)$

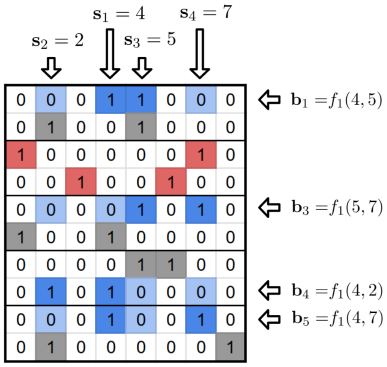
Assume we set $\Sigma = \{1, \ldots, 8\}$ and that $\mathbf{s} = (4, 2, 5, 7)$. Every coordinate of $\mathbf{b}$ will equal the corresponding function evaluation, except with probability α. The parameter α is supposed to be close to 1, but for this example we take $\alpha \approx 1/5$.

3. The public key will be a $(m' \leq |\Sigma|^{k/3}, |\Sigma|, k)$-matrix $\mathbf{H}$ defined as follows. For all $i \in [m]$ and for all tuples $(\sigma_1, \ldots, \sigma_k) \in \Sigma^k$ such that $f_i(\sigma_1, \ldots, \sigma_k) = \mathbf{b}_i$, append the length-$|\Sigma|$ *indicator vector* for the tuple $(\sigma_1, \ldots, \sigma_k)$ as a new row in $\mathbf{H}$. Define $\mathbf{H}$ to have m' rows and n' columns. Notice that when producing $\mathbf{H}$, we ignore all structure coming from the original matrix $\mathbf{G}$. So if we instead sampled $\mathbf{b}$ at random, $\mathbf{H}$ would be a truly random matrix; this is useful for proving security.



An example matrix $\mathbf{H}$ produced using the matrix $\mathbf{G}$ from above. We assume that every equation $\mathbf{b}_i = f_i(\sigma_1, \sigma_2)$ has exactly two solutions. For an actual instantiation of the PKE scheme, the number of solutions is approximately $|\Sigma|^k/|\Gamma| = |\Sigma|^{k/4}$ for each constraint.

4. By construction of $\mathbf{H}$, a row-induced column-permuted submatrix $\mathbf{G}'$ of $\mathbf{G}$ will appear within $\mathbf{H}$, where $\mathbf{G}'$ is formed by deleting each row of $\mathbf{G}$ independently with probability α. The secret key will be a mapping ζ which records the location of $\mathbf{G}'$ in $\mathbf{H}$. Put differently, we know that there exists a subset of the rows of $\mathbf{H}$ which generate a *punctured version* of the original code $\{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$, and ζ allows us to identify (in the correct order) the coordinates which correspond to this punctured version.



An annotated version of the matrix $\mathbf{H}$. Every row that came from a corrupted constraint pair $(\mathbf{b}_i, f_i)$ is marked in red, every row that represents a solution to $\mathbf{b}_i = f_i(\mathbf{s}_{N_\mathbf{G}(i,1)}, \mathbf{s}_{N_\mathbf{G}(i,2)})$ is marked in blue, and all other rows are marked in gray. The column-permuted submatrix $\mathbf{G}'$ is marked in light and dark blue. Note that within this light/dark blue submatrix, the order of the columns from $\mathbf{G}$ is now: (2nd, 1st, 3rd, 4th), and the second row of $\mathbf{G}$ was deleted.

Encryption

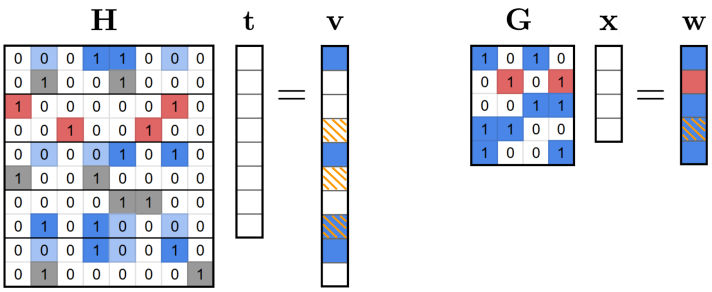
1. To encrypt a bit $b = 0$, sample a ciphertext $\mathbf{v} \in \mathbb{F}_2^{m'}$ as in the planted distribution in the High Corruption kXOR Conjecture. That is, sample $\mathbf{t} \in \mathbb{F}_2^{n'}$ at random and then sample $\mathbf{v} \in \mathbb{F}_2^{m'}$ as

$$\mathbf{v}_i = \begin{cases} \text{Random element of } \mathbb{F}_2, & \text{with probability } \beta = 1 - o(1). \\ \mathbf{t}_{N_\mathbf{H}(i,1)} + \ldots + \mathbf{t}_{N_\mathbf{H}(i,k)}, & \text{otherwise.} \end{cases}$$

2. To encrypt a bit $b = 1$, sample a ciphertext $\mathbf{v} \in \mathbb{F}_2^{m'}$ as in the null distribution in the High Corruption kXOR Conjecture, i.e. sample $\mathbf{v}$ at random.

Decryption

1. Use the secret key ζ to extract a subvector $\mathbf{w} \in \{\mathbb{F}_2 \cup \text{“?”}\}^m$ of the ciphertext $\mathbf{v}$, where “?” is a special erasure symbol. If we encrypted a bit $b = 0$, then $\mathbf{w}$ should be a codeword from $\{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$, but where each entry is erased (i.e. replaced with “?”) with probability α, and the remaining entries are corrupted with probability β. The erasure rate α comes from the probability that a row of $\mathbf{G}$ is deleted when constructing $\mathbf{G}'$. The corruption rate β comes from the corruption rate at encryption time.



Above, we give a worked example of setting up the decryption process (in the case of an encrypted bit $b = 0$) using the matrices $\mathbf{G}$ and $\mathbf{H}$ from key generation. The coordinates of $\mathbf{v}$ that are corrupted are marked in orange, and the coordinates of $\mathbf{v}$ corresponding to $\mathbf{G}$ are marked in blue. We extract a codeword $\mathbf{w}$ from $\mathbf{v}$, which is shown on the right. The (known) erased coordinates are marked in red, and the (unknown) corrupted coordinates are marked in orange[8].

2. Run algorithm Distinguish on $\mathbf{w}$, and determine the encrypted bit based on the result.

Intuitively speaking, decryption is possible because *both* the matrix $\mathbf{G}$ *and* the predicate

$$\mathbf{w}_i = \mathbf{x}_{N_{\mathbf{G}}(i,1)} + \ldots + \mathbf{x}_{N_{\mathbf{G}}(i,k)}$$

are of "low complexity," so we can distinguish noisy vectors $\mathbf{w}$ from totally random vectors. In the proof of security, however, we show that an adversary must either solve a CSP where the predicates are of maximum complexity (i.e. break the High Corruption LARP-CSP Conjecture), or where the factor graph is of maximum complexity (i.e. break the High Corruption kXOR Conjecture).

Remark 1.6. Applebaum [App12] used an algorithm similar to our key generation algorithm to show hardness of approximation for the densest k-subhypergraph problem, assuming hardness of Goldreich's PRG. One important difference is that Applebaum constructed the hypergraph instance by *sub-sampling* local pre-images for a (collection of) Goldreich instances, while we construct the public key $\mathbf{H}$ by listing *all* local pre-images for a LARP-CSP instance. This distinction is one key factor that ensures correctness of our decryption algorithm.

Instantiating an $(\alpha = 1 - o(1), \beta = 1 - o(1))$-Decodable Expanding Code. In Section 5.1 we give an explicit randomized algorithm for MatrixGen that outputs a $(1 - o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix $\mathbf{G}$ that generates a subcode of a sufficiently low-rate Reed-Muller code:

Theorem 1.7. *There is an explicit randomized algorithm $\mathsf{K}_{c_k,c_m}(1^n)$ parameterized by two constants c_k, c_m satisfying $c_m \geq 2$ and $c_k \geq c_m + 1$, that runs in time $2^{O((\log n)^{c_m})}$ and outputs a matrix $\mathbf{G}$, such that:*

1. *$\mathbf{G}$ is an (m, n, k)-matrix, where $m = 2^{(\lceil \log n \rceil)^{c_m}}$ and $k = (\lceil \log n \rceil)^{c_k}$.*
2. *The code $\{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$ is a subcode of the Reed-Muller code RM$((\lceil \log n \rceil)^{c_m}, (\lceil \log n \rceil)^2)$.*
3. *With $1 - o(1)$ probability over the coins of K_{c_k,c_m}, $\mathbf{G}$ is $(1 - o(1), n^{1-o(1)})$-expanding.*

Remark 1.8. The running time can also be written as $m^{O(1)}$, where m is the height of the output matrix $\mathbf{G}$. In fact the running time will be dominated by the time required to write the output $\mathbf{G}$.

The above theorem improves upon the expander graphs constructed by Oliveira, Santhanam, and Tell [OST18] in the following ways: First and foremost, our construction is given by an explicit *uniform* algorithm, whereas Oliveira, Santhanam, and Tell were only able to prove the *existence* of expander graphs that suit their needs. Our algorithm is also quite simple, whereas the construction of Oliveira, Santhanam, and Tell relies on machinery related to the Nisan-Wigderson pseudorandom generator [NW94].

We leverage an algorithm of Saptharishi, Shpilka, and Volk [SSV16] to make the algorithm Distinguish. Their results shows that any Reed-Muller code of sufficiently low rate can be decoded from a $1 - o(1)$ fraction of errors, which after some massaging implies a distinguisher in the sense required for our code.

Remember that by Theorem 1.5 this $(1-o(1), 1-o(1))$-decodable expanding code immediately implies a PKE scheme from $1 - o(1)$ corruption CSPs, from which we deduce Theorem 1.3.

[8]Note that, of course, it would be impossible to distinguish $\mathbf{w}$ from random in the drawn example, because with only 3 non-corrupted coordinates in $\mathbf{w}$ that depend on 3 random inputs in $\mathbf{x}$, in fact all non-erased coordinates of our drawn $\mathbf{w}$ will be distributed totally randomly. In an actual decryption instance, the number of non-corrupted coordinates of $\mathbf{w}$ will far exceed the dimension of $\mathbf{x}$, since the code $\mathbf{G}$ is of vanishing rate.

1.4 Discussion

In this work we present a radically different approach for building public key encryption, motivated by a desire to explore and understand what kinds of hardness conjectures can be used to (explicitly) build public-key encryption. We believe that algorithmic research on LARP-CSPs will be fruitful for the algorithms and complexity communities, because of its connections to cryptography, Goldreich's PRG, and planted hypergraph problems.

Related Public Key Encryption Schemes. Our public-key encryption scheme, especially the way that we plant a trapdoor, is very notably different from all previous public-key encryption schemes. The closest relatives to our scheme appear to be those given in [ABW10, BKR23, GHJS25]. In our paper and all of these papers, there is a similar 2-phase construction that combines a planting assumption at key generation with a planting assumption at encryption time. But the similarities stop here, since our key generation and decryption algorithms make critical use of a novel error correcting code introduced in this paper. We note that the McEliece cryptosystem [McE78] and its relatives do make critical use of a hidden error correcting code. *But unlike in the McEliece-type cryptosystems,* we have a concrete and natural hardness conjecture related to the average-case hardness of CSPs (the LARP-CSP Conjecture) which allows us to argue that our error correcting code *can* be hidden, and indeed hidden within the generator matrix of a *random* linear code.

Open Questions. Our work opens several new lines of research with respect to algorithm design, lower bounds, and coding theory. We believe that the following questions are of highest priority:

1. Can we further populate the list of computational lower bounds for LARP-CSPs? What are the parameter regimes in which LARP-CSPs appear to be computationally intractable?
2. Is there a unified framework which captures the known attacks on Goldreich's PRG? In this paper we give the first framework which captures the attacks of Oliveira, Santhanam, and Tell [OST18], but we believe that significantly more research is warranted.
3. Does there exist a $(1-o(1), 1-o(1))$-decodable expanding code in the *polynomial stretch regime*? If so, this would imply a PKE scheme with plausibly subexponential security via Theorem 1.5.
4. What is the minimal level of mathematical structure required to build PKE solely from the hardness of $1-o(1)$ corruption CSPs?
5. Is it possible to build public-key encryption solely from *unstructured* hardness conjectures like our LARP-CSP conjecture?

2 Preliminaries

We use $[n]$ to denote the set $\{1, \ldots, n\}$. All vectors are column vectors unless stated otherwise. The phrase "sample at random" is shorthand for "sample uniformly at random." $\text{Binom}(n, p)$ is the binomial distribution with n trials and success probability p. For a field $\mathbb{F}$ and two vectors $\mathbf{b}, \mathbf{v} \in \mathbb{F}^m$, we use $\text{dist}(\mathbf{b}, \mathbf{v})$ to denote the number of coordinates on which $\mathbf{b}$ and $\mathbf{v}$ differ. For a vector $\mathbf{b} \in \mathbb{F}^m$ and a subset $\mathcal{L} \subset \mathbb{F}^m$, we use $\text{dist}(\mathbf{b}, \mathcal{L})$ to denote the minimum over all $\mathbf{v} \in \mathcal{L}$ of $\text{dist}(\mathbf{b}, \mathbf{v})$. All logarithms are taken base 2.

Erasures and Corruptions. We use the term "subjected to erasures of rate α" to refer to the random process which replaces each entry of a given vector with an erasure symbol "?", independently with probability α. For binary vectors, this is equivalent to transmission over the *binary erasure channel* (BEC) with erasure rate α.

We use the term "subjected to corruptions of rate β" to refer to the random process which replaces each entry of a given vector with a random value, independently with probability β. For binary vectors, this is equivalent to transmission over the *binary symmetric channel* (BSC) with error rate $\frac{\beta}{2}$. The reason for this discrepancy is that the BSC error rate is actually the *bit flip probability*. Just as a corruption rate of 1 replaces the entire vector with noise, a BSC with error rate $1/2$ replaces the entire vector with noise. The reason we use "corruption rate" is that it generalizes directly to non-binary alphabets.

Matrices and Expanders. For a vector $\mathbf{v} \in \mathbb{F}_2^n$, let $\mathsf{hw}(\mathbf{v})$ be its Hamming weight, i.e. the number of nonzero entries. For a matrix $\mathbf{M} \in \mathbb{F}_2^{m \times n}$, let $\mathbf{M}_i \in \mathbb{F}_2^n$ be the row of $\mathbf{M}$ indexed by i. For vectors $\mathbf{v}_1, \ldots \mathbf{v}_t \in \mathbb{F}_2^n$, we use $\bigvee_{i \in t} \mathbf{v}_i$ to denote their component-wise logical-OR. We now give some definitions related to sparse matrices and their expansion properties.

Definition 2.1 ((m, n, k)-Matrix)**.** *An (m, n, k)-matrix is a matrix $\mathbf{M} \in \mathbb{F}_2^{m \times n}$ where each row has* exactly *k nonzero entries, i.e. $\mathsf{hw}(\mathbf{M}_i) = k$ for all $i \in [m]$.*

Definition 2.2 (Neighbor Function $N_{\mathbf{M}}(i, j)$)**.** *For a matrix $\mathbf{M} \in \mathbb{F}_2^{m \times n}$, we use $N_{\mathbf{M}}(i, j)$ to denote the column index of the jth nonzero entry of $\mathbf{M}_i$.*

Now we define what it means for a matrix over $\mathbb{F}_2$ to be an expander.[9]

Definition 2.3 ((γ, t)-Expander)**.** *A matrix $\mathbf{M} \in \mathbb{F}_2^{m \times n}$ is a (γ, t)-expander if and only if, for all subsets $\mathcal{S} \subseteq [m]$ with $1 \leq |\mathcal{S}| \leq t$,*

$$\mathsf{hw}\left(\bigvee_{i \in \mathcal{S}} \mathbf{M}_i\right) \geq \gamma \cdot \sum_{i \in \mathcal{S}} \mathsf{hw}(\mathbf{M}_i).$$

If $\mathbf{M}$ is an (m, n, k)-matrix, this definition is equivalent to

$$\mathsf{hw}\left(\bigvee_{i \in \mathcal{S}} \mathbf{M}_i\right) \geq \gamma k |\mathcal{S}|.$$

2.1 Public Key Encryption

We start with a definition that captures public key encryption (PKE) schemes which are only "mildly" correct and "mildly" secure.

Definition 2.4 (See Definition 8 in [HR05])**.** *A $(\kappa(\lambda), \eta(\lambda))$-*secure public key encryption scheme *is a triple of probabilistic polynomial time algorithms* $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ *such that*

1. $\mathsf{KeyGen}(1^\lambda)$ *outputs a pair of strings* $(\mathsf{pk}, \mathsf{sk})$.
2. $\mathsf{Enc}(1^\lambda, \mathsf{pk}, b \in \{0, 1\})$ *outputs a string* ct.
3. $\mathsf{Dec}(1^\lambda, \mathsf{sk}, \mathsf{ct})$ *outputs a bit $b' \in \{0, 1\}$.*

[9] This definition is agnostic to the number of nonzero entries in each row, but we will only apply it to (m, n, k)-matrices.

4. ($\kappa(\lambda)$-correctness) *For a random bit* $b \in \{0,1\}$*,*

$$\Pr\Big[(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); \mathsf{Dec}(1^\lambda,\mathsf{sk},\mathsf{Enc}(1^\lambda,\mathsf{pk},b)) = b\Big] > \frac{1+\kappa(\lambda)}{2},$$

where the probability is taken over the internal coins of KeyGen, Enc, *and* Dec*.*

5. ($\eta(\lambda)$-security) *For all poly*(λ) *size non-uniform algorithms* $\mathcal{A}$*, for a random bit* $b \in \{0,1\}$*,*

$$\Pr\Big[(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); \mathcal{A}(\mathsf{pk},\mathsf{Enc}(1^\lambda,\mathsf{pk},b)) = b\Big] < \frac{1+\eta(\lambda)}{2},$$

where the probability is taken over the internal coins of KeyGen, Enc, *and* Dec*.*

Remark 2.5. Item 5 is equivalent to bounding the advantage of an algorithm as follows:[10] For all poly(λ) size non-uniform algorithms $\mathcal{A}$,

$$\Big|\Pr\Big[(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); \mathcal{A}(\mathsf{pk},\mathsf{Enc}(1^\lambda,\mathsf{pk},1)) = 1\Big] - \Pr\Big[(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); \mathcal{A}(\mathsf{pk},\mathsf{Enc}(1^\lambda,\mathsf{pk},0)) = 1\Big]\Big| < \eta(\lambda).$$

PKE schemes satisfying the standard notion of correctness and security can be defined as follows.

Definition 2.6 (Based on [GM84])**.** *A* semantically secure public key encryption scheme *is a triple of probabilistic polynomial time algorithms* (KeyGen, Enc, Dec) *such that for all* $1-\kappa = \eta \geq \lambda^{-O(1)}$*,* (KeyGen, Enc, Dec) *is a* (κ,η)*-secure public key encryption scheme.*

We make critical use of a theorem by Holenstein and Renner which allows us to amplify from a (κ,η)-secure PKE scheme to a standard PKE scheme, assuming κ is sufficiently large with respect to η.

Theorem 2.7 (See Theorem 6 in [HR05])**.** *Let* κ,η *be any* constants *satisfying* $\kappa^2 > \eta$*. Then there is a black box reduction from any* (κ,η)*-secure public key encryption scheme to a semantically secure public key encryption scheme.*

2.2 Coding Theory

The *minimum distance* of a code is the number of nonzero entries in a minimum-weight nonzero codeword. The *dual* of a code $\mathcal{C}$ is the code consisting of all vectors $\mathbf{c}'$ such that, for all $\mathbf{c} \in \mathcal{C}$, the inner product $<\mathbf{c}',\mathbf{c}>$ satisfies $<\mathbf{c}',\mathbf{c}>=0$. The dual to a linear code is always a linear code.

Reed-Muller Codes. We define $\binom{d}{\leq r} := \sum_{i=0}^{r}\binom{d}{\leq r}$. $\mathcal{M}(d,r)$ denotes the set of all monomials of degree at most r over d variables in $\mathbb{F}_2$. Observe that $|\mathcal{M}(d,r)| = \binom{d}{\leq r}$, because we always include the constant monomial "1" in $\mathcal{M}(d,r)$. We consider each monomial $M \in \mathcal{M}(d,r)$ as a function, so that we can write $M(\mathbf{p})$ to denote the evaluation of M on a given point $\mathbf{p} \in \mathbb{F}_2^d$.

Definition 2.8 (Reed-Muller (RM) Code [Ree53, Mul54])**.** *The Reed-Muller code* $RM(d,r)$ *is a linear code consisting of the length-*2^d *evaluation vectors for all polynomials of degree at most* r *over* $\mathbb{F}_2^d$*.*

[10] The proof is folklore and uses elementary linear algebraic manipulations.

Equivalently, we can define the RM code using an "evaluation matrix" $\mathbf{E} \in \mathbb{F}_2^{2^d \times \binom{d}{\leq r}}$. Each row of $\mathbf{E}$ is indexed by a different point $\mathbf{p} \in \mathbb{F}_2^d$, each column is indexed by a distinct monomial $M \in \mathcal{M}(d, r)$, and we set $\mathbf{E}_{\mathbf{p},M} = M(\mathbf{p})$. Now we can write

$$\mathrm{RM}(d, r) = \{\mathbf{E}\mathbf{x} : \mathbf{x} \in \mathbb{F}_2^{\binom{d}{\leq r}}\},$$

where $\mathbf{x}$ is can be interpreted as the coefficient vector for a degree $\leq r$ polynomial.

We need the following basic lemmas, both of which are well known.

Lemma 2.9 (See e.g. [ASY20])**.** *The minimum distance of RM(d, r) is 2^{d-r}.*

Lemma 2.10 (See e.g. [ASY20])**.** *The dual code to RM(d, r) is RM$(d, d - r - 1)$.*

Saptharishi, Shpilka, and Volk showed that Reed-Muller codes of sufficiently low rate can be decoded from a $1 - o(1)$ fraction of random corruptions.

Theorem 2.11 (Special case of Corollary 14 in [SSV16])**.** *There is a function $\gamma(d) \geq 1 - o(1), \gamma(d) \leq 1 - d^{-O(1)}$ and a poly(2^d) time algorithm D that outputs a vector in RM$(d, d^{1/3})$ and behaves as follows. Pick a random codeword $\mathbf{c} \in$ RM$(d, d^{1/3})$, and sample a noisy version $\mathbf{c}'$ of $\mathbf{c}$:*

$$\mathbf{c}'_i = \begin{cases} \text{Random element of } \mathbb{F}_2, & \text{with probability } \gamma(d). \\ \mathbf{c}_i, & \text{otherwise.} \end{cases}$$

Then

$$\Pr\left[\mathsf{D}(\mathbf{c}') = \mathbf{c}\right] \geq 1 - o(1).$$

Remark 2.12. Technically speaking, Corollary 14 in [SSV16] is for a fixed number of random errors. But as pointed out by the authors, results for a fixed number of errors translate to the Bernoulli error case with vanishing parameter loss. (For a discussion of this phenomenon, see [ASW15].) The Bernoulli error case is then equivalent to our corruption model after rescaling parameters.

Notational Guidance. Throughout the paper, we use slightly non-standard notation for our codes; for example, we denote the Reed-Muller code as $\mathrm{RM}(d, r)$ instead of $\mathrm{RM}(m, r)$, we use n for the code dimension, and we use m for the block length. This is so that we can refer to our cryptographic subproblems as involving "$m \times n$" matrices. In general we will default to cryptographic notation.

3 Our Conjectures

We build a public key encryption scheme whose security is implied by the following conjectures when instantiated with corruption rates of $\alpha = 1 - o(1)$ and $\beta = 1 - o(1)$. Recall from Section 1.1 and Section 1.2 that we have a variety of lower bounds suggesting the conjectures in fact hold against all $\exp\left\{n^{o(1)}\right\}$ size adversaries. See the introduction for a more detailed discussion.

Conjecture 3.1 (LARP-CSP(α) Conjecture)**.** *Let $\mathbf{H}$ be any $(1 - o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix, where $k = (\log n)^{\Theta(1)}$ and $m \leq n^{o(k)}$. Let Σ, Γ be any alphabets satisfying $|\Sigma| = (nm)^{\log^{\Theta(1)}(nm)}$ and $|\Gamma| \leq |\Sigma|^{3k/4}$. Sample m random functions $f_i : \Sigma^k \to \Gamma$, and let $\mathcal{F}$ be the set of all f_i. Then no poly$(|\Sigma|^k)$ size algorithm can distinguish, with advantage more than $1/4$, between the following two distributions.*

1. *$(\mathbf{H}, \mathcal{F}, \mathbf{b})$, where $\mathbf{b} \in \Gamma^m$ is sampled at random.*

2. $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, *where* $\mathbf{b} \in \Gamma^m$ *is sampled as follows. Sample* $\mathbf{s} \in \Sigma^n$ *at random and set*

$$\mathbf{b}_i = \begin{cases} \textit{Random element of } \Gamma, & \textit{with probability } \alpha. \\ f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)}), & \textit{otherwise.} \end{cases}$$

Conjecture 3.2 (kXOR(β) Conjecture)**.** *Let* $\mathbf{H}$ *be a random* (m, n, k)*-matrix, where* $k = (\log n)^{\Theta(1)}$ *and* $m \leq n^{k/3}$*. Then no poly*(m) *size algorithm can distinguish, with advantage more than* $1/4$*, between the following two distributions.*

1. $(\mathbf{H}, \mathbf{b})$, *where* $\mathbf{b} \in \mathbb{F}_2^m$ *is sampled at random.*
2. $(\mathbf{H}, \mathbf{b})$, *where* $\mathbf{b} \in \mathbb{F}_2^m$ *is sampled as follows. Sample* $\mathbf{s} \in \mathbb{F}_2^n$ *at random and set*

$$\mathbf{b}_i = \begin{cases} \textit{Random element of } \mathbb{F}_2, & \textit{with probability } \beta. \\ \mathbf{s}_{N_{\mathbf{H}}(i,1)} + \ldots + \mathbf{s}_{N_{\mathbf{H}}(i,k)}, & \textit{otherwise.} \end{cases}$$

4 From Decodable Expanding Codes to PKE

4.1 The General Framework

We first define an (α, β)*-decodable expanding code*, which consists of a code generation algorithm MatrixGen and a distinguishing algorithm Distinguish. The name comes from the fact that the error correcting code will have a *low density generator matrix* (LDGM), and additionally this generator matrix will be strongly expanding. Intuitively, we mandate that Distinguish can differentiate between (i) random vectors subjected to erasures of rate α, and (ii) codewords subjected to erasures of rate α and corruptions (on the non-erased coordinates) of rate β.

Definition 4.1 ((α, β)-Decodable Expanding Code)**.** [11] *A tuple* $(\mathsf{MatrixGen}, \mathsf{Distinguish}, \alpha, \beta)$*, where* MatrixGen *and* Distinguish *are algorithms, and* α *and* β *are parameters. The code generation algorithm* MatrixGen *has the following properties:*

1. $\mathsf{MatrixGen}(1^m)$ *runs in* $m^{O(1)}$ *time and outputs an* (m, n, k)*-matrix* $\mathbf{G}$*, where* $k = \log^{\Theta(1)} m$ *and* n *satisfies* $m = n^{o(k)}$*. We interpret* $\mathbf{G}$ *as the generator matrix for a linear code over* $\mathbb{F}_2$*.*
2. *With* $1 - o(1)$ *probability over the coins of* $\mathsf{MatrixGen}(1^n)$*,* $\mathbf{G}$ *is a* $(1 - o(1), n^{1-o(1)})$*-expander.*

The distinguishing algorithm Distinguish *has the following properties:*

1. $\mathsf{Distinguish}(1^m, \mathbf{G}, \mathbf{w})$ *takes as input an* (m, n, k)*-matrix* $\mathbf{G}$ *produced by* MatrixGen *along with a vector* $\mathbf{w} \in \{\mathbb{F}_2 \cup \text{“?”}\}^m$*, runs in time* $m^{O(1)}$*, and outputs a bit* $b \in \{0, 1\}$*.*
2. *Pick a random codeword* $\mathbf{c} \in \{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$*, and sample a noisy version* $\mathbf{c}' \in \{\mathbb{F}_2 \cup \text{“?”}\}^m$ *of* $\mathbf{c}$*:*

$$\mathbf{c}'_i = \begin{cases} \textit{“?”}, & \textit{with probability } \alpha. \\ \textit{Random element of } \mathbb{F}_2, & \textit{with probability } (1-\alpha)\beta. \\ \mathbf{c}_i, & \textit{otherwise.} \end{cases}$$

[11] As mentioned in the introduction, we actually only need a distinguisher, which is simple to construct when given a decoder.

Also sample a random vector $\mathbf{r} \in \{\mathbb{F}_2 \cup \text{“?”}\}^m$*:*

$$\mathbf{r}_i = \begin{cases} \text{“?”}, & \text{with probability } \alpha. \\ \text{Random element of } \mathbb{F}_2, & \text{otherwise.} \end{cases}$$

Then

$$\Pr\big[\mathsf{Distinguish}(1^n, \mathbf{G}, \mathbf{c}') = 0\big] \geq 1 - o(1) \quad \textit{and} \quad \Pr[\mathsf{Distinguish}(1^n, \mathbf{G}, \mathbf{r}) = 1] \geq 1 - o(1),$$

where the probability ranges over the choice of $\mathbf{G}$*,* $\mathbf{c}'$*, and* $\mathbf{r}$*, as well as the coins of* Distinguish*.*

Given any (α, β)-decodable expanding code with internal parameters m, n, k, we define a public key encryption scheme as follows. Fix two alphabets Σ, Γ satisfying $|\Sigma| = (nm)^{\log^{\Theta(1)}(nm)}$ and $|\Gamma| = |\Sigma|^{3k/4}$. The security parameter λ is set to the maximum of $|\Sigma|^k$, the running time of MatrixGen, and the running time of Distinguish. It will be clear from the exposition that all algorithms run in time $|\Sigma|^{O(k)}$, so $\lambda = |\Sigma|^{O(k)}$.

Remark 4.2. Strictly speaking, we would like to instantiate the PKE scheme by choosing λ first and then setting the other parameters. Because the relationship between $n, m, k, |\Sigma|, |\Gamma|$, and λ is well defined, we can simply fix a target value λ^* and then calculate values for the parameters so that the actual security parameter λ is within a small slack factor of the target λ^*. Notice that $n, m, |\Sigma|$, and $|\Gamma|$ will all be of the form $2^{\log^c \lambda}$ for different constants $c > 0$, and k will be of the form $\log^{\Theta(1)} \lambda$.

Key Generation.

$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$:

1. $\mathbf{G} \leftarrow \mathsf{MatrixGen}(1^m)$.
2. Sample functions $f_1, \ldots, f_m : \Sigma^k \to \Gamma$ at random.
3. Sample $\mathbf{b} \in \Gamma^m$ as follows. Sample $\mathbf{s} \in \Sigma^n$ at random, and set

$$\mathbf{b}_i = \begin{cases} \text{Random element of } \Gamma, & \text{with probability } \alpha. \\ f_i(\mathbf{s}_{N_{\mathbf{G}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{G}}(i,k)}), & \text{otherwise.} \end{cases}$$

4. Define the set

$$\mathcal{X} \coloneqq \left\{(\sigma_1, \ldots, \sigma_k) \in \Sigma^k : \text{ there exists } i \in [m] \text{ such that } f_i(\sigma_1, \ldots, \sigma_k) = \mathbf{b}_i\right\}.$$

5. Delete all tuples from $\mathcal{X}$ that contain two or more copies of the same symbol.
6. If at least one of the following holds, output $(\mathsf{pk}, \mathsf{sk}) \coloneqq (\perp, \perp)$ and abort.
   (a) There exist indices $i, j \in [n]$ with $i \neq j$ such that $\mathbf{s}_i = \mathbf{s}_j$.
   (b) $|\mathcal{X}| > |\Sigma|^{k/3}$.
7. Define $\mathbf{H} \in \mathbb{F}_2^{|\mathcal{X}| \times |\Sigma|}$ as follows.
   (a) Label each column by a distinct $\sigma \in \Sigma$.

(b) Label each row by a distinct $(\sigma_1, \ldots, \sigma_k) \in \mathcal{X}$.

(c) Set $\mathbf{H}_{((\sigma_1,\ldots,\sigma_k),\sigma)} = 1$ if and only if σ equals one of $\sigma_1, \ldots, \sigma_k$.

8. Modify $\mathbf{H}$ as follows.

   (a) Append random rows with exactly k nonzero entries, until the height of $\mathbf{H}$ is exactly $|\Sigma|^{k/3}$.

   (b) Randomly permute the rows of $\mathbf{H}$.

9. Define a function $\zeta : [m] \to \left[|\Sigma|^{k/3}\right] \cup$ "?" as follows.

   (a) If $\mathbf{b}_i$ was *not* set to a random value in Step 3, then $\zeta(i)$ is set to the index of the row representing $(\mathbf{s}_{N_{\mathbf{G}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{G}}(i,k)})$ in $\mathbf{H}$. Such a row is guaranteed to exist by Step 6a.

   (b) Otherwise, $\zeta(i)$ is set to $\perp$.

10. Output $(\mathsf{pk}, \mathsf{sk}) \coloneqq (\mathbf{H}, \zeta)$.

Encryption.

$\mathsf{ct} \leftarrow \mathsf{Enc}(1^\lambda, \mathsf{pk}, b \in \{0,1\})$:

1. If $\mathsf{pk} = \perp$, output $\mathsf{ct} \coloneqq \perp$ and abort.
2. Otherwise, parse pk as $\mathbf{H} \in \mathbb{F}_2^{m' \times n'}$, where $m' = |\Sigma|^{k/3}$ and $n' = |\Sigma|$.
3. If $b = 0$, sample $\mathbf{v} \in \mathbb{F}_2^{m'}$ as follows. Sample $\mathbf{t} \in \mathbb{F}_2^{n'}$ at random and set

$$\mathbf{v}_i = \begin{cases} \text{Random element of } \mathbb{F}_2, & \text{with probability } \beta. \\ \mathbf{t}_{N_{\mathbf{H}}(i,1)} + \ldots + \mathbf{t}_{N_{\mathbf{H}}(i,k)}, & \text{otherwise.} \end{cases}$$

4. If $b = 1$, then sample $\mathbf{v} \in \mathbb{F}_2^{m'}$ at random.
5. Output $\mathsf{ct} \coloneqq \mathbf{v}$.

Decryption.

$b' \leftarrow \mathsf{Dec}(1^\lambda, \mathsf{sk}, \mathsf{ct})$:

1. If $\mathsf{ct} = \perp$, output $b' \coloneqq \perp$ and abort.
2. Otherwise, parse ct as $\mathbf{v}$ and sk as ζ.
3. Define a vector $\mathbf{w} \in \{\mathbb{F}_2 \cup \text{"?"}\}^m$ as follows.

   (a) If $\zeta(i) \neq \perp$, then $\mathbf{w}_i = \mathbf{v}_{\zeta(i)}$.

   (b) Otherwise, $\mathbf{w}_i =$ "?".

4. Output $b' \coloneqq \mathsf{Distinguish}(1^m, \mathbf{G}, \mathbf{w})$.

Now our plan in Sections 4.2 and 4.3 is to argue κ-correctness and η-security. In particular, we get a

$(0.99, 0.9)$-secure[12] PKE scheme, assuming Conjecture 3.1 (LARP-CSP) holds with corruption parameter α and Conjecture 3.2 (kXOR) holds with corruption parameter β. This implies a semantically secure PKE scheme by applying Theorem 2.7. In Section 5 we give an explicit $(1-o(1), 1-o(1))$-decodable expanding code, along with the concrete PKE scheme it implies.

4.2 Correctness

In this subsection we show that $b' = b$ with sufficiently high probability. More precisely, we prove:

Lemma 4.3 (Correctness)**.** *Let $\kappa = 0.99$,[13] and assume λ is sufficiently large. For a random bit $b \in \{0,1\}$,*

$$\Pr\Big[(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); \mathsf{Dec}(1^\lambda, \mathsf{sk}, \mathsf{Enc}(1^\lambda, \mathsf{pk}, b)) = b\Big] > \frac{1+\kappa}{2},$$

where the probability is taken over the internal coins of KeyGen, Enc, *and* Dec.

At its core, the proof is quite simple. The matrix $\mathbf{H}$ should contain a *homomorphic copy* of the original matrix $\mathbf{G}$ (but with a $1 - o(1)$ fraction of the rows deleted), and the secret function ζ describes where this homomorphic copy lies. Because encryption of a bit $b = 0$ amounts to computing a noisy linear sample using the matrix $\mathbf{H}$, a small portion of the ciphertext will actually correspond to a noisy codeword from the linear code $\{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$. The function ζ allows us to extract this noisy codeword, and from here the distinguishing algorithm for the code allows us to determine whether a bit $b = 0$ or a bit $b = 1$ was encrypted. Unfortunately, there are several technical details to handle, e.g. the failure conditions in the key generation algorithm.

Proof of Lemma 4.3. We start by bounding the probability that KeyGen aborts, i.e. at least one of the two failure conditions in Step 6 is satisfied. Below we argue that it is unlikely for two symbols of the secret $\mathbf{s} \in \Sigma^n$ to be the same.

Claim 4.4. *The probability that there exists an index pair $i, j \in [n]$ with $i \neq j$ such that $\mathbf{s}_i = \mathbf{s}_j$ is $o(1)$.*

Proof. There are $O(m^2)$ pairs i, j, and the probability that a fixed pair collides is $1/|\Sigma|$. A Markov bound shows that the probability of at least one collision occurring across all pairs is $O(m^2|\Sigma|^{-1})$. Because m and $|\Sigma|$ grow with λ, and $|\Sigma| = m^{\omega(1)}$, this quantity is bounded as $o(1)$. □

Now we argue that the set of tuples $\mathcal{X}$ is unlikely to be too large.

Claim 4.5. *The probability that $|\mathcal{X}| > |\Sigma|^{k/3}$ is $o(1)$.*

Proof. Recall that we defined $\mathcal{X}$ by taking the set

$$\Big\{(\sigma_1, \ldots, \sigma_k) \in \Sigma^k : \text{ there exists } i \in [m] \text{ such that } f_i(\sigma_1, \ldots, \sigma_k) = \mathbf{b}_i\Big\}$$

and then deleting all tuples that contain two or more copies of the same symbol. Define random variables

$$Y_i = \Big|\Big\{(\sigma_1, \ldots, \sigma_k) \in \Sigma^k : f_i(\sigma_1, \ldots, \sigma_k) = \mathbf{b}_i\Big\}\Big|,$$

and let $Y = \sum_{i \in m} Y_i$. Observe that we always have $Y \geq |\mathcal{X}|$, so it will be sufficient to show that $Y \leq |\Sigma|^{k/3}$ with probability $1 - o(1)$. If $\mathbf{b}_i$ was set to $f_i(\mathbf{s}_{N_\mathbf{G}(i,1)}, \ldots, \mathbf{s}_{N_\mathbf{G}(i,k)})$, we know by the randomness of f_i that Y_i is distributed as

$$Y_i \sim 1 + \text{Binom}(|\Sigma|^k - 1, |\Gamma|^{-1}).$$

[12] Technically, the PKE scheme will be $(1 - o(1), \frac{3}{4} + o(1))$-secure, but the constants 0.99 and 0.9 are easier to work with.

[13] In the proof we actually show that we can take $\kappa = 1 - o(1)$.

In the case that $\mathbf{b}_i$ was set to a random value, we again know by the randomness of f_i that

$$Y_i \sim \text{Binom}(|\Sigma|^k, |\Gamma|^{-1}).$$

In both cases, because $|\Gamma| = |\Sigma|^{3k/4}$, a Chernoff bound shows that

$$\Pr[Y_i \geq 2\mathbb{E}[Y_i]] \leq \Pr\left[Y_i \geq 2(1 + |\Sigma|^{k/4})\right] < |\Sigma|^{-\omega(1)}.$$

Applying a Markov bound over these events and assuming λ is sufficiently large (and therefore m, $|\Sigma|$ are sufficiently large), the probability that a there exists a single Y_i which is at least $2(1 + |\Sigma|^{k/4})$ is bounded as $o(1)$. But this event is a necessary condition for Y to exceed $|\Sigma|^{k/3}$, because otherwise

$$Y = \sum_{i \in [m]} Y_i \leq m \cdot 2(1 + |\Sigma|^{k/4}) < |\Sigma|^{k/3},$$

again assuming λ is sufficiently large and using that $|\Sigma| = m^{\omega(1)}$. □

Before arguing that Distinguish allows us to decrypt successfully, first consider a modified PKE scheme where we remove Step 5 and Step 6 in algorithm KeyGen. We now demonstrate that the distributions over vectors $\mathbf{w}$ in algorithm Dec are within $o(1)$ statistical distance of the distributions over vectors for which we assumed that Distinguish is an effective distinguisher.

Claim 4.6. *Consider a PKE scheme identical to the one in Section 4.1, but where we remove Step 5 and Step 6 in algorithm* KeyGen. *Then $\zeta(i) = \perp$ independently with probability α for all i.*

Proof. Observe that $\mathbf{b}_i$ is set to a random value in Γ independently with probability α, for all i. Now in Step 9, we set $\zeta(i) = \perp$ whenever this event occurs. Otherwise, $\zeta(i)$ will map to the appropriate row index of $\mathbf{H}$. Because we removed Step 5 and Step 6, such a row index always exists. □

Claim 4.7. *Consider a PKE scheme identical to the one in Section 4.1, but where we remove Step 5 and Step 6 in algorithm* KeyGen. *Sample* $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$ *and* $\mathsf{ct} \leftarrow \mathsf{Enc}(1^\lambda, \mathsf{pk}, 0)$*, and then run* $\mathsf{Dec}(1^\lambda, \mathsf{sk}, \mathsf{ct})$*. Over the random coins of* KeyGen *and* Enc*, the distribution over vectors $\mathbf{w}$ in algorithm* Dec *is within $o(1)$ statistical distance of the following distribution over vectors $\mathbf{c}'$. Sample $\mathbf{c} \in \{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$ at random, and then sample $\mathbf{c}'$:*

$$\mathbf{c}'_i = \begin{cases} \text{“?”}, & \text{with probability } \alpha. \\ \text{Random element of } \mathbb{F}_2, & \text{with probability } (1-\alpha)\beta. \\ \mathbf{c}_i, & \text{otherwise.} \end{cases}$$

Proof. We first argue that $\mathbf{w}$ will always be a noisy version of some codeword from $\mathcal{C} = \{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$. Consider the matrix $\mathbf{G}'$ defined as follows:

1. If $\zeta(i) \neq \perp$, then $\mathbf{G}'_i = \mathbf{H}_{\zeta(i)}$.
2. Otherwise, $\mathbf{G}'_i = \mathbf{0}^{|\Sigma|}$.

Then delete all columns of $\mathbf{G}'$ which are *not* indexed by some symbol $\mathbf{s}_i$ of the secret $\mathbf{s}$ as produced in KeyGen, and let the width of $\mathbf{G}'$ be n''. Notice that by definition of ζ, we have that $\mathbf{G}'$ is simply a copy of $\mathbf{G}$ where:

1. Some rows are replaced with $\mathbf{0}$.

2. The columns are permuted.

3. Some of the columns might be "merged" by taking their coordinate-wise locical-OR. This happens whenever there exist two indices $i, j \in [n]$ such that $i \neq j$ but $\mathbf{s}_i = \mathbf{s}_j$.

By Claim 4.4, we know that with probability $1 - o(1)$, none of the columns are merged. In this case the linear code $\mathcal{C}' = \{\mathbf{G}'\mathbf{x} : \mathbf{x} \in \mathbb{F}_2^{n''}\}$ is the same as $\mathcal{C} = \{\mathbf{G}\mathbf{x} : \mathbf{x} \in \mathbb{F}_2^n\}$, but where a fixed set of coordinates is set to zero. By definition of Enc and Dec, $\mathbf{w}$ will be a corrupted version of a random codeword from $\mathcal{C}'$, where the coordinates that are always set to 0 will be replaced with "?".

Now we argue that the erasure and corruption probabilities for $\mathbf{w}$ exactly match those in the statement of the claim (regardless of whether some columns of $\mathbf{G}$ are merged):

1. By Claim 4.2, we know that $\zeta(i) = \perp$ independently with probability α for all i. By the construction of $\mathbf{w}$, we know that $\mathbf{w}_i =$ "?" if and only if $\zeta(i) = \perp$, so the probability that $\mathbf{w}_i =$ "?" is exactly α, independently for each coordinate.

2. By definition of Enc, we know that every non-erased coordinate of $\mathbf{w}$ will be set to a random value in $\mathbb{F}_2$ independently with probability β. Combining this with the erasure probability, we have that the probability a coordinate is *not* erased but *is* corrupted will be exactly $(1-\alpha)\beta$, independently for each coordinate.

3. Otherwise, the coordinate will be left un-erased and un-corrupted.

□

Claim 4.8. *Consider a PKE scheme identical to the one in Section 4.1, but where we remove Step 5 and Step 6 in algorithm* KeyGen. *Sample* $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$ *and* $\mathsf{ct} \leftarrow \mathsf{Enc}(1^\lambda, \mathsf{pk}, 1)$*, and then run* $\mathsf{Dec}(1^\lambda, \mathsf{sk}, \mathsf{ct})$. *Over the random coins of* KeyGen *and* Enc*, the distribution over vectors* $\mathbf{w}$ *in algorithm* Dec *is identical to the distribution over vectors* $\mathbf{r} \in \{\mathbb{F}_2 \cup \text{"?"}\}^m$ *sampled as follows:*

$$\mathbf{r}_i = \begin{cases} \text{"?"}, & \text{with probability } \alpha. \\ \text{Random element of } \mathbb{F}_2, & \text{otherwise.} \end{cases}$$

Proof. Almost identical to the proof of Claim 4.7. The only difference is to observe that, when $b = 1$, $\mathsf{Enc}(1^\lambda, \mathsf{pk}, b)$ will set the ciphertext to a uniformly random vector in $\mathbb{F}_2^{m'}$. This means that the distribution over vectors $\mathbf{w}$ is identical to the distribution over vectors $\mathbf{r}$, as opposed to just being within $o(1)$ statistical distance. □

Now consider the actual PKE scheme from Section 4.1, i.e. we include Step 5 and Step 6. By Claim 4.4 and Claim 4.5, the probability that KeyGen aborts is $o(1)$. If KeyGen does not abort, we still have that $\mathbf{w}_i =$ "?" if and only if $\zeta(i) = \perp$, and furthermore $\zeta(i) = \perp$ if and only if $\mathbf{b}_{\zeta(i)}$ was set to a random value in Step 3 of algorithm KeyGen. Putting all of this together with Claim 4.7 and Claim 4.8, we have:

1. In the case that we encrypted a bit $b = 0$, the distribution on vectors $\mathbf{w}$ is within $o(1)$ statistical distance of the distribution on vectors $\mathbf{c}'$ for which

$$\Pr\left[\mathsf{Distinguish}(1^n, \mathbf{G}, \mathbf{c}') = 0\right] \geq 1 - o(1)$$

2. In the case that we encrypted a bit $b = 1$, the distribution on vectors $\mathbf{w}$ is within $o(1)$ statistical distance of the distribution on vectors $\mathbf{r}$ for which

$$\Pr[\mathsf{Distinguish}(1^n, \mathbf{G}, \mathbf{r}) = 1] \geq 1 - o(1)$$

So for the true PKE scheme,

$$\Pr\Big[(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); \mathsf{Dec}(1^\lambda,\mathsf{sk},\mathsf{Enc}(1^\lambda,\mathsf{pk},b)) = b\Big] \geq 1 - o(1).$$

Taking λ sufficiently large completes the proof of correctness. □

4.3 Security

We use a standard hybrid argument to demonstrate security, by invoking Conjecture 3.1 (LARP-CSP) and Conjecture 3.2 (kXOR).

Lemma 4.9 (Security)**.** *Let $\eta = 0.9$,[14] and let λ be sufficiently large. Assume that Conjecture 3.1 (LARP-CSP) holds with corruption rate α, and Conjecture 3.2 (kXOR) holds with corruption rate β. Then for all $\lambda^{O(1)}$-size non-uniform algorithms $\mathcal{A}$,*

$$\Big|\Pr\Big[(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); \mathcal{A}(\mathsf{pk},\mathsf{Enc}(1^\lambda,\mathsf{pk},1)) = 1\Big] - \Pr\Big[(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); \mathcal{A}(\mathsf{pk},\mathsf{Enc}(1^\lambda,\mathsf{pk},0)) = 1\Big]\Big| < \eta$$

Proof. Define the following hybrids.

1. $\mathsf{H}_0(1^\lambda)$:
   (a) Invoke $\mathsf{KeyGen}(1^\lambda)$ to obtain a public key pk.
   (b) Invoke $\mathsf{Enc}(1^\lambda,\mathsf{pk},0)$ to obtain a ciphertext ct.
   (c) Output $(\mathsf{pk},\mathsf{ct})$.
2. $\mathsf{H}_{0,\$}(1^\lambda)$:
   (a) Define KeyGen' to be an algorithm identical to KeyGen, but where Step 3 in KeyGen sets $\mathbf{b}$ to a random vector in Γ^m. Invoke $\mathsf{KeyGen}'(1^\lambda)$ to obtain a public key pk.
   (b) Invoke $\mathsf{Enc}(1^\lambda,\mathsf{pk},0)$ to obtain a ciphertext ct.
   (c) Output $(\mathsf{pk},\mathsf{ct})$.
3. $\mathsf{H}_{1,\$}(1^\lambda)$:
   (a) As before, define KeyGen' to be an algorithm identical to KeyGen, but where Step 3 in KeyGen sets $\mathbf{b}$ to a random vector in Γ^m. Invoke $\mathsf{KeyGen}'(1^\lambda)$ to obtain a public key pk.
   (b) Invoke $\mathsf{Enc}(1^\lambda,\mathsf{pk},1)$ to obtain a ciphertext ct.
   (c) Output $(\mathsf{pk},\mathsf{ct})$.
4. $\mathsf{H}_1(1^\lambda)$:
   (a) Invoke $\mathsf{KeyGen}(1^\lambda)$ to obtain a public key pk.
   (b) Invoke $\mathsf{Enc}(1^\lambda,\mathsf{pk},1)$ to obtain a ciphertext ct.
   (c) Output $(\mathsf{pk},\mathsf{ct})$.

Now we argue that an adversary has advantage less than 0.3 to distinguish between each pair of hybrids.

[14] We could actually take $\eta = \frac{3}{4} + o(1)$; the choice $\eta = 0.9$ just makes the arguments a bit simpler.

Claim 4.10. *Assume that Conjecture 3.1 (LARP-CSP) holds with corruption rate α. Then for all sufficiently large λ, for all $\lambda^{O(1)}$-size non-uniform algorithms $\mathcal{A}$,*

$$\left|\Pr\left[\mathcal{A}(\mathsf{H}_0(1^\lambda)) = 1\right] - \Pr\left[\mathcal{A}(\mathsf{H}_{0,\$}(1^\lambda)) = 1\right]\right| < 0.3$$

Proof. Suppose for contradiction that there exists a $\lambda^{O(1)}$-size non-uniform algorithm $\mathcal{A}$ such that

$$\left|\Pr\left[\mathcal{A}(\mathsf{H}_0(1^\lambda)) = 1\right] - \Pr\left[\mathcal{A}(\mathsf{H}_{0,\$}(1^\lambda)) = 1\right]\right| \geq 0.3.$$

By assumption on $\mathsf{MatrixGen}$, $\mathbf{G}$ will be a $(1-o(1), n^{1-o(1)})$-expander with probability $1-o(1)$. Thus

$$\Big|\Pr\left[\mathcal{A}(\mathsf{H}_0(1^\lambda)) = 1 \middle| \mathbf{G} \text{ is a } (1-o(1), n^{1-o(1)})\text{-expander}\right] - \Pr\left[\mathcal{A}(\mathsf{H}_{0,\$}(1^\lambda)) = 1 \middle| \mathbf{G} \text{ is a } (1-o(1), n^{1-o(1)})\text{-expander}\right]\Big| \geq 0.3 - o(1),$$

and by the averaging principle there must exist a specific $(1-o(1), n^{1-o(1)})$-expander $\mathbf{G}^*$ such that

$$\Big|\Pr\left[\mathcal{A}(\mathsf{H}_0(1^\lambda)) = 1 \middle|\ \mathsf{H}_0(1^\lambda) \text{ samples } \mathbf{G} = \mathbf{G}^*\right] - \Pr\left[\mathcal{A}(\mathsf{H}_{0,\$}(1^\lambda)) = 1 \middle|\ \mathsf{H}_{0,\$}(1^\lambda) \text{ samples } \mathbf{G} = \mathbf{G}^*\right]\Big| \geq 0.3 - o(1).$$

By definition of H_0 and $\mathsf{H}_{0,\$}$, we can reduce the decision problem in Conjecture 3.1 with matrix $\mathbf{G}^*$, corruption parameter α, and parameters $n, m, k, |\Sigma|, |\Gamma|$ to the problem of differentiating between the outputs of $\mathsf{H}_0(1^\lambda)$ and $\mathsf{H}_{0,\$}(1^\lambda)$, where both $\mathsf{H}_0(1^\lambda)$ and $\mathsf{H}_{0,\$}(1^\lambda)$ are conditioned on sampling $\mathbf{G} = \mathbf{G}^*$. Thus $\mathcal{A}$ implies a $\lambda^{O(1)} = |\Sigma|^{O(k)}$ size algorithm for the problem in Conjecture 3.1 with distinguishing advantage at least $0.3 - o(1)$. Assuming λ is sufficiently large, this exceeds $1/4$, which is a contradiction. □

Claim 4.11. *Assume that Conjecture 3.2 (kXOR) holds with corruption rate β. Then for all $\lambda^{O(1)}$-size non-uniform algorithms $\mathcal{A}$,*

$$\left|\Pr\left[\mathcal{A}(\mathsf{H}_{0,\$}(1^\lambda)) = 1\right] - \Pr\left[\mathcal{A}(\mathsf{H}_{1,\$}(1^\lambda)) = 1\right]\right| < 0.3$$

Proof. Suppose for contradiction that there exists a $\lambda^{O(1)}$ size non-uniform algorithm $\mathcal{A}$ such that

$$\left|\Pr\left[\mathcal{A}(\mathsf{H}_{0,\$}(1^\lambda)) = 1\right] - \Pr\left[\mathcal{A}(\mathsf{H}_{1,\$}(1^\lambda)) = 1\right]\right| \geq 0.3.$$

We argue that this directly gives an algorithm contradicting Conjecture 3.2.

The only difference between the two hybrids is the sampling procedure for $\mathbf{v}$ in algorithm Enc. In fact, conditioning both $\mathsf{H}_{0,\$}$ and $\mathsf{H}_{1,\$}$ on not aborting, the distribution $(\mathsf{pk}, \mathsf{ct}) \sim \mathsf{H}_{0,\$}(1^\lambda)$ is exactly distribution (2) in Conjecture 3.2, and the distribution $(\mathsf{pk}, \mathsf{ct}) \sim \mathsf{H}_{1,\$}(1^\lambda)$ is exactly distribution (1), where both distributions are with respect to an $m' \times n'$ matrix $\mathbf{H}$ with $m' = (n')^{k/3}$. So given $\mathcal{A}$, we construct a $\lambda^{O(1)} = |\Sigma|^{O(k)} = (m')^{O(1)}$ size distinguisher $\mathcal{B}(\mathbf{H}, \mathbf{b})$ for Conjecture 3.2 by picking the best choice of randomness for the following procedure:

1. Run $\mathsf{H}_{0,\$}(1^\lambda)$. If the output is $(\perp, \perp)$ then return $\mathcal{A}(\perp, \perp)$.
2. Otherwise, return $\mathcal{A}(\mathbf{H}, \mathbf{b})$.

$\mathcal{B}$ has advantage at least $0.3 > 1/4$, which is a contradiction. □

Claim 4.12. *Assume that Conjecture 3.1 (LARP-CSP) holds with corruption rate α. Then for all sufficiently large λ, for all $\lambda^{O(1)}$-size non-uniform algorithms $\mathcal{A}$,*

$$\left|\Pr\left[\mathcal{A}(\mathsf{H}_{1,\$}(1^\lambda)) = 1\right] - \Pr\left[\mathcal{A}(\mathsf{H}_1(1^\lambda)) = 1\right]\right| < 0.3$$

Proof. Identical to the proof of Claim 4.10. □

Composing Claims 4.10, 4.11, and 4.12 completes the proof of security. □

4.4 Conversion to a Semantically Secure PKE

Putting together the ingredients from Sections 4.1, 4.2, and 4.3, we get a semantically secure PKE scheme.

Theorem 4.13. *Suppose we have an (α, β)-decodable expanding code. Then assuming Conjecture 3.1 (LARP-CSP) holds with corruption rate α, and Conjecture 3.2 (kXOR) holds with corruption rate β, there is a semantically secure public key encryption scheme.*

Proof. By Lemmas 4.3 and 4.9, the PKE scheme in Section 4.1 instantiated with an (α, β)-decodable expanding code is a $(0.99, 0.9)$-secure PKE scheme. Because $0.99^2 = 0.9801 > 0.9$, an application of Theorem 2.7 gives us a semantically secure PKE scheme. □

5 Instantiating a $(1 - o(1), 1 - o(1))$-Decodable Expanding Code

In this section we construct an (α, β)-decodable expanding code, where $\alpha = \beta = 1 - o(1)$. As discussed in Section 4.4, this gives a semantically secure PKE scheme if we assume that Conjecture 3.1 and Conjecture 3.2 both hold.

5.1 Sampling the Generator Matrix

Here we give a simple probabilistic algorithm that runs in time $m^{O(1)}$ and outputs a generator matrix $\mathbf{G} \in \mathbb{F}_2^{m \times n}$ for an expanding code $\mathcal{C} = \{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$, such that $\mathcal{C}$ is a subcode of $\mathrm{RM}(\log m, \log^2 n)$. This will be the algorithm $\mathsf{MatrixGen}$ for our (α, β)-decodable expanding code.

Theorem 1.7 (Restated)**.** *There is an explicit randomized algorithm $\mathsf{K}_{c_k,c_m}(1^n)$ parameterized by two constants c_k, c_m satisfying $c_m \geq 2$ and $c_k \geq c_m + 1$, that runs in time $2^{O((\log n)^{c_m})}$ and outputs a matrix $\mathbf{G}$, such that:*

1. *$\mathbf{G}$ is an (m, n, k)-matrix, where $m = 2^{(\lceil \log n \rceil)^{c_m}}$ and $k = (\lceil \log n \rceil)^{c_k}$.*
2. *The code $\{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$ is a subcode of the Reed-Muller code $\mathrm{RM}((\lceil \log n \rceil)^{c_m}, (\lceil \log n \rceil)^2)$.*
3. *With $1 - o(1)$ probability over the coins of K_{c_k,c_m}, $\mathbf{G}$ is $(1 - o(1), n^{1-o(1)})$-expanding.*

Proof. We propose the following algorithm. (Recall that all logarithms are taken base 2.)

$\mathbf{G} \leftarrow \mathsf{K}_{c_k,c_m}(1^n)$:

1. Set $k := (\lceil \log n \rceil)^{c_k}$, $m := 2^{(\lceil \log n \rceil)^{c_m}}$, $w := \lfloor \log(n/k) \rfloor$
2. Sample kw random degree-$\lceil \log n \rceil$ polynomials

$$g^{(1,1)}, \ldots, g^{(1,w)}, g^{(2,1)}, \ldots, g^{(k,w)} : \mathbb{F}_2^{\log m} \to \mathbb{F}_2.$$

3. For all $i \in [k]$, we construct a matrix $\mathbf{M}^{(i)} \in \mathbb{F}_2^{m \times 2^w}$ as follows.
   (a) Each row is indexed by a point $\mathbf{p} \in \mathbb{F}_2^{\log m}$
   (b) Each column is indexed by a point $\mathbf{q} \in \mathbb{F}_2^w$.
   (c) Set $M^{(i)}_{\mathbf{p},\mathbf{q}} = 1$ if and only if $g^{(i,j)}(\mathbf{p}) = \mathbf{q}_j$ for all $j \in [w]$, or equivalently set $M^{(i)}_{\mathbf{p},\mathbf{q}} = 1$ if and only if the concatenation of the evaluations $g^{(i,1)}(\mathbf{p}) \| \ldots \| g^{(i,w)}(\mathbf{p})$ equals $\mathbf{q}$.
4. Set $\mathbf{G} = \left[\mathbf{M}^{(1)} \| \ldots \| \mathbf{M}^{(k)}\right]$, and then append zero columns until the width is exactly n.
5. Output $\mathbf{G}$.

By construction, the algorithm runs in $2^{O((\log n)^{c_m})}$ time, and $\mathbf{G}$ is a matrix over $\mathbb{F}_2$ of width n and height m. To show that every row has exactly k nonzero entries, it suffices to show that each matrix $\mathbf{M}^{(i)}$ has exactly one nonzero entry per row. This follows because, for every point $\mathbf{p} \in \mathbb{F}_2^{\log m}$, there is exactly one point $\mathbf{q} \in \mathbb{F}_2^w$ such that $g^{(i,j)}(\mathbf{p}) = \mathbf{q}_j$ for all $j \in [w]$.

We now argue that $\mathbf{G}$ generates a subcode of $\mathrm{RM}((\lceil \log n \rceil)^{c_m}, (\lceil \log n \rceil)^2)$, which amounts to showing that every column is the truth table for a degree $\leq (\lceil \log n \rceil)^2$ polynomial. Every column coming not coming from a submatrix $M^{(i)}$ is zero and thus of degree zero, so we can ignore these columns. Now fix any choice of i and examine the submatrix $M^{(i)}$. Observe that Step 3c is equivalent to defining

$$\mathbf{M}^{(i)}_{\mathbf{p},\mathbf{q}} = \prod_{j \in [w]} \left(g^{(i,j)}(\mathbf{p}) + \mathbf{q}_j + 1 \right).$$

If we fix any column, this amounts to fixing a choice of point $\mathbf{q}$, and allowing $\mathbf{p}$ to range over all points in $\mathbb{F}_2^m$. This gives the truth table for a polynomial of degree at most

$$\sum_{j \in [w]} \mathrm{degree}(g^{(i,j)}).$$

Every $g^{(i,j)}$ is of degree at most $\lceil \log n \rceil$ by construction, and $w := \lfloor \log(n/k) \rfloor < \lceil \log n \rceil$. So the degree is upper bounded by $w \lceil \log n \rceil < (\lceil \log n \rceil)^2$.

All that remains is to show that, with probability $1 - o(1)$, $\mathbf{G}$ is a $(1 - o(1), n^{1-o(1)})$-expander. Intuitively, the proof will have three parts:

1. Show that a *random* matrix with the same parameters will be a $(1 - o(1), n^{1-o(1)})$-expander.
2. Show that, *on a local level,* the matrix $\left[\mathbf{M}^{(1)} \| \ldots \| \mathbf{M}^{(k)}\right]$ is *statistically random*.
3. Compose these results via a union bound to show that $\left[\mathbf{M}^{(1)} \| \ldots \| \mathbf{M}^{(k)}\right]$, and hence $\mathbf{G}$, is a $(1 - o(1), n^{1-o(1)})$-expander.

Claim 5.1. *Let $\mathbf{R}^{(1)}, \ldots, \mathbf{R}^{(k)} \in \mathbb{F}_2^{t \times 2^w}$ be sampled at random, conditioned on each matrix having exactly 1 nonzero entry per row, and let $\mathbf{R} := \left[\mathbf{R}^{(1)} \| \ldots \| \mathbf{R}^{(k)}\right]$. Then assuming n is sufficiently large and $1 \leq t \leq 2^w/(\lceil \log n \rceil)^2$, with probability at least $1 - 2^{-kt/\sqrt{\lceil \log n \rceil}}$,*

$$\mathsf{hw}\left(\bigvee_{i \in [t]} \mathbf{R}_i \right) > \left(1 - 1/\sqrt{\lceil \log n \rceil}\right) \cdot k \cdot t.$$

Proof. It suffices to bound the probability that

$$W = \mathsf{hw}\left(\bigvee_{i\in[t]} \mathbf{R}_i\right) \le \left(1 - 1/\sqrt{\lceil \log n\rceil}\right)\cdot k\cdot t.$$

Consider sampling each row of each $\mathbf{R}^{(j)}$ one-by-one. The probability that a new row $\mathbf{R}_i^{(j)}$ *does not* increase W is exactly the probability that the nonzero entry in $\mathbf{R}_i^{(j)}$ falls into the same column as a previously sampled nonzero entry. Because each $\mathbf{R}^{(j)}$ has t rows, the probability that this event occurs is at most $t/2^w$, independently for each $i\in[t]$ and $j\in[k]$. There are kt choices for i,j, so $W \le \left(1-1/\sqrt{\lceil\log n\rceil}\right)\cdot k\cdot t$ if and only if this event occurs for at least $kt/\sqrt{\lceil\log n\rceil}$ rows $\mathbf{R}_i^{(j)}$. This probability is upper bounded as

$$\begin{aligned}
&\binom{kt}{kt/\sqrt{\lceil\log n\rceil}}\cdot (t/2^w)^{kt/\sqrt{\lceil\log n\rceil}} \\
<&\left(3\sqrt{\lceil\log n\rceil}\right)^{kt/\sqrt{\lceil\log n\rceil}}\cdot (t/2^w)^{kt/\sqrt{\lceil\log n\rceil}} && \text{(Standard approximation } \tbinom{x}{y} < (3x/y)^y\text{)}\\
\le&\left(3\sqrt{\lceil\log n\rceil}\right)^{kt/\sqrt{\lceil\log n\rceil}}\cdot \left((\lceil\log n\rceil)^{-2}\right)^{kt/\sqrt{\lceil\log n\rceil}} && (t\le 2^w/(\lceil\log n\rceil)^2)\\
<&\lceil\log n\rceil^{kt/\sqrt{\lceil\log n\rceil}}\cdot \left((\lceil\log n\rceil)^{-2}\right)^{kt/\sqrt{\lceil\log n\rceil}} && \text{(Assuming } n \text{ is sufficiently large)}\\
=&\lceil\log n\rceil^{-kt/\sqrt{\lceil\log n\rceil}}\\
<&2^{-kt/\sqrt{\lceil\log n\rceil}} && \text{(Assuming } n \text{ is sufficiently large)}
\end{aligned}$$

□

Claim 5.2. *Fix a subset $\mathcal{T}\subset\mathbb{F}_2^{\log m}$ of size $1\le|\mathcal{T}|\le 2^w/(\lceil\log n\rceil)^2$, and then sample $\mathbf{M}^{(1)},\dots,\mathbf{M}^{(k)}$ as in the algorithm K_{c_k,c_m}, denoting their concatenation as $\mathbf{M}=\left[\mathbf{M}^{(1)}\|\dots\|\mathbf{M}^{(k)}\right]$. Assuming n is sufficiently large, with probability at least $1-2^{-k|\mathcal{T}|/\sqrt{\lceil\log n\rceil}}$,*

$$\mathsf{hw}\left(\bigvee_{\mathbf{p}\in[t]}\mathbf{M}_{\mathbf{p}}\right) > \left(1-1/\sqrt{\lceil\log n\rceil}\right)\cdot k\cdot|\mathcal{T}|.$$

Proof. Recall that every polynomial $g^{(i,j)}$ is a random degree-$\lceil\log n\rceil$ polynomial $g^{(i,j)}:\mathbb{F}_2^{\log m}\to\mathbb{F}_2$. Thus the length-$m$ vector $\mathbf{v}^{(i,j)}$ defined as $\mathbf{v}^{(i,j)}_{\mathbf{p}} = g^{(i,j)}(\mathbf{p})$, where $\mathbf{p}$ ranges over all points in $\mathbb{F}_2^{\log m}$, will be a random member of the Reed-Muller code $\mathrm{RM}(\log m,\lceil\log n\rceil)$. Now by Lemma 2.10, the dual code to $\mathrm{RM}(\log m,\lceil\log n\rceil)$ is $\mathrm{RM}(\log m,\log m-\lceil\log n\rceil-1)$, and by Lemma 2.9 the minimum distance of this dual code will be $2^{\lceil\log n\rceil+1}>n$. This means that if we fix at most n points and then sample a random polynomial $g^{(i,j)}$ of degree $\lceil\log n\rceil$, the evaluation of $g^{(i,j)}$ on each of these points will be independently drawn from $\mathrm{Ber}(1/2)$.

Because $2^w/(\lceil\log n\rceil)^2 = 2^{\lfloor\log(n/k)\rfloor}/(\lceil\log n\rceil)^2 < n$, this means that for a fixed subset $\mathcal{T}\subset\mathbb{F}_2^{\log m}$ of size at most $2^w/(\lceil\log n\rceil)^2$, the evaluations of all polynomials $g^{(i,j)}$ on points in $\mathcal{T}$ will be uniformly random. As a result, the distribution over matrices $\mathbf{M}$ restricted to the rows indicated by $\mathcal{T}$ will be uniformly random, conditioned on each sub-matrix $\mathbf{M}^{(i)}$ having exactly one nonzero entry per row. Now the proof is completed by applying Claim 5.1. □

From here we just need to union bound over all choices of subsets $\mathcal{T}$.

Claim 5.3. *With probability $1-o(1)$, for all sufficiently large n, the matrix $\mathbf{G}$ produced in algorithm K_{c_k,c_m} is a $(1-1/\sqrt{\lceil\log n\rceil}, 2^w/(\lceil\log n\rceil)^2)$-expander.*

Proof. Fix a size parameter $1 \le t \le 2^w/(\lceil\log n\rceil)^2)$. There are

$$\binom{m}{t} \le m^t$$

size-t subsets of rows in $\mathbf{G}$, and by Claim 5.2 each subset violates expansion with probability at most

$$2^{-kt/\sqrt{\lceil\log n\rceil}}.$$

Union bounding over all choices of rows, the probability that at least one size-t subset violates expansion is at most

$$\begin{aligned}
&m^t \cdot 2^{-kt/\sqrt{\lceil\log n\rceil}} \\
=&\left(2^{(\lceil\log n\rceil)^{c_m}}\right)^t \cdot 2^{-((\lceil\log n\rceil)^{c_k})t/\sqrt{\lceil\log n\rceil}} && (m \coloneqq 2^{(\lceil\log n\rceil)^{c_m}} \text{ and } k \coloneqq (\lceil\log n\rceil)^{c_k}) \\
=&2^{t\left((\lceil\log n\rceil)^{c_m} - (\lceil\log n\rceil)^{c_k}/\sqrt{\lceil\log n\rceil}\right)} \\
\le&2^{t\left((\lceil\log n\rceil)^{c_m} - (\lceil\log n\rceil)^{c_m+1}/\sqrt{\lceil\log n\rceil}\right)}. && (c_k \ge c_m+1) \\
\le&2^{t\left((\lceil\log n\rceil)^{c_m} - (\lceil\log n\rceil)^{c_m+1/2}\right)}.
\end{aligned}$$

Assuming n is sufficiently large, this is less than

$$2^{-\frac{t}{2}(\lceil\log n\rceil)^{c_m+1/2}}.$$

A final union bound over all choices of t completes the proof. □

The above shows that $\mathbf{G}$ is a $(1-1/\sqrt{\lceil\log n\rceil}, 2^w/(\lceil\log n\rceil)^2)$-expander with probability $1-o(1)$. Observe that

$$\begin{aligned}
2^w/(\lceil\log n\rceil)^2) =&2^{\lfloor\log(n/k)\rfloor}/(\lceil\log n\rceil)^2 \\
\ge&\Omega(2^{\log(n/k)}/(\log n)^2) \\
=&\Omega(n/(\log n)^{2+c_k}) \\
=&n/(\log n)^{O(1)} \\
=&n^{1-o(1)}.
\end{aligned}$$

Thus $\mathbf{G}$ is a $(1-o(1), n^{1-o(1)})$-expander with probability $1-o(1)$, as desired. □

5.2 The Distinguishing Algorithm

We use the Reed-Muller decoding algorithm of Saptharishi, Shpilka, and Volk [SSV16] as a starting point for the algorithm Distinguish in our (α,β)-decodable expanding code.

Theorem 2.11 (Due to [SSV16], Restated). *There is a function $\gamma(d) \geq 1 - o(1), \gamma(d) \leq 1 - d^{-O(1)}$ and a poly(2^d) time algorithm D that outputs a vector in RM$(d, d^{1/3})$ and behaves as follows. Pick a random codeword $\mathbf{c} \in$ RM$(d, d^{1/3})$, and sample a noisy version $\mathbf{c}'$ of $\mathbf{c}$:*

$$\mathbf{c}'_i = \begin{cases} \textit{Random element of } \mathbb{F}_2, & \textit{with probability } \gamma(d). \\ \mathbf{c}_i, & \textit{otherwise.} \end{cases}$$

Then

$$\Pr\left[\mathsf{D}(\mathbf{c}') = \mathbf{c}\right] \geq 1 - o(1).$$

The conversion to a distinguisher goes as follows.

Lemma 5.4. *There is a function $\mu(d) \geq 1 - o(1), \mu(d) \leq 1 - d^{-O(1)}$ and a poly(2^d) time algorithm E that outputs a vector in RM$(d, d^{1/3})$ and behaves as follows. Let $m = 2^d$. Pick a random codeword $\mathbf{c} \in$ RM$(d, d^{1/3})$, and sample a noisy version $\mathbf{c}' \in \{\mathbb{F}_2 \cup \text{“?”}\}^m$ of $\mathbf{c}$:*

$$\mathbf{c}'_i = \begin{cases} \textit{“?”}, & \textit{with probability } \mu. \\ \textit{Random element of } \mathbb{F}_2, & \textit{with probability } (1-\mu)\mu. \\ \mathbf{c}_i, & \textit{otherwise.} \end{cases}$$

Also sample a random vector $\mathbf{r} \in \{\mathbb{F}_2 \cup \text{“?”}\}^m$:

$$\mathbf{r}_i = \begin{cases} \textit{“?”}, & \textit{with probability } \mu. \\ \textit{Random element of } \mathbb{F}_2, & \textit{otherwise.} \end{cases}$$

Then

$$\Pr\left[\mathsf{E}(\mathbf{c}') = 0\right] \geq 1 - o(1) \quad \textit{and} \quad \Pr[\mathsf{E}(\mathbf{r}) = 1] \geq 1 - o(1).$$

Proof. Let $\gamma(d) = 1 - o(1)$ be as in Theorem 2.11, and set $\mu(d) = 1 - \sqrt{1 - \gamma(d)} = 1 - o(1)$. Observe that the decoder D from that theorem doubles as a decoder for a noisy version $\mathbf{c}' \in \{\mathbb{F}_2 \cup \text{“?”}\}^m$ of a random codeword $\mathbf{c} \in \text{RM}(d, d^{1/3})$ sampled as follows:

$$\mathbf{c}'_i = \begin{cases} \text{“?”}, & \text{with probability } \mu. \\ \text{Random element of } \mathbb{F}_2, & \text{with probability } (1-\mu)\mu. \\ \mathbf{c}_i, & \text{otherwise.} \end{cases}$$

This is because we can replace every erasure symbol “?” with a random element of $\mathbb{F}_2$ and then apply D on the resulting vector, which will have a corruption rate of

$$\begin{aligned} \mu + (1-\mu)\mu &= \left(1 - \sqrt{1-\gamma}\right) + \sqrt{1-\gamma}\left(1 - \sqrt{1-\gamma}\right) \\ &= 1 - \sqrt{1-\gamma} + \sqrt{1-\gamma} - \left(\sqrt{1-\gamma}\right) \cdot \left(\sqrt{1-\gamma}\right) \\ &= 1 - (1-\gamma) \\ &= \gamma \end{aligned}$$

Now we describe how to convert the decoder into a distinguisher. Since $\gamma(d) \leq 1 - d^{-O(1)} = 1 - (\log n)^{-O(1)}$, a concentration bound shows that there is a cutoff value z^* satisfying the following conditions.

1. Let $\mathbf{c}$ be a random codeword from $\mathrm{RM}(d, d^{1/3})$, and let $\mathbf{c}'$ be the vector $\mathbf{c}$ subjected to corruptions of rate γ. Then $\mathbf{c}'$ disagrees with $\mathbf{c}$ on less than z^* coordinates with probability $1 - o(1)$. Consequently $\mathsf{D}(\mathbf{c}')$ disagrees with $\mathbf{c}'$ on less than than z^* coordinates with probability $1 - o(1)$.

2. Let $\mathbf{r} \in \mathbb{F}_2^m$ be a random vector. With probability $1 - o(1)$, for all $\mathbf{c} \in \mathrm{RM}(d, d^{1/3})$, $\mathbf{r}$ disagrees with $\mathbf{c}$ on more than z^* coordinates. Since D outputs a vector in $\mathrm{RM}(d, d^{1/3})$, we have that with probability $1 - o(1)$, $\mathsf{D}(\mathbf{r})$ disagrees with $\mathbf{r}$ on more than than z^* coordinates.

So the distinguisher just invokes D, counts the number of coordinates on which the input vector and output vector disagree, and accepts or rejects by performing a threshold test on this value. □

5.3 Putting it Together

Combining Theorem 4.13 with Theorems 1.7 and 5.4, we have our PKE scheme.

Theorem 1.3 (Restated)**.** *Suppose that Conjecture 1.1 (High Corruption LARP-CSP) and Conjecture 1.2 (High Corruption kXOR) both hold. Then there is a semantically secure public key encryption scheme.*

Proof. Recall that Conjecture 1.2 is simply Conjecture 3.2 instantiated with any choice of corruption rate $\beta(n) = 1 - o(1)$, and similarly Conjecture 1.1 is simply Conjecture 3.1 instantiated with any choice of corruption rate $\alpha(n) = 1 - o(1)$. By Theorem 4.13, it suffices to exhibit an (α, β)-decodable expanding code, where $\alpha(n) = \beta(n) = 1 - o(1)$.

Fix $c_k = 7$ and $c_m = 6$. The matrices $\mathbf{G}$ sampled in our $(1 - o(1), 1 - o(1))$-decodable expanding code will be of width n and height $m = 2^{O((\log n)^6)}$, and each row will have $k = (\lceil \log n \rceil)^7$ nonzero entries. By Theorem 1.7, we have a $2^{O((\log n)^6)} = m^{O(1)}$ time algorithm $\mathsf{K}_{7,6}(1^m)$ that can be used as the algorithm $\mathsf{MatrixGen}$. Furthermore, for every choice of internal coins, the (m, n, k)-matrix $\mathbf{G}$ output by $\mathsf{K}_{7,6}(1^m)$ will generate a subcode $\mathcal{C} = \{\mathbf{Gx} : \mathbf{x} \in \mathbb{F}_2^n\}$ of $\mathrm{RM}(d = (\lceil \log n \rceil)^6, d^{1/3} = (\lceil \log n \rceil)^2)$. We can thus use algorithm E from Lemma 5.4 directly as the algorithm $\mathsf{Distinguish}$, and the running time of this algorithm is $2^{O(d)} = m^{O(1)}$. The error rate and corruption rate that this algorithm can tolerate both tend towards 1 as d increases; because d increases with n, they are both $1 - o(1)$. □

6 Evidence for the LARP-CSP Conjecture

In this section we give a variety of algorithmic lower bounds supporting Conjecture 3.1 (LARP-CSP). In fact, all of our lower bounds will be for the *corruption-free* version. We formalize this version of the decision problem as follows.

Problem 1. *Let $\mathbf{H}$ be any $(1 - o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix, where $k = (\log n)^{\Theta(1)}$ and $m \leq n^{o(k)}$. Let Σ, Γ be any alphabets satisfying $|\Sigma| = (nm)^{\log^{\Theta(1)}(nm)}$ and $|\Gamma| \leq |\Sigma|^{3k/4}$. Sample m random functions $f_i : \Sigma^k \to \Gamma$, and let $\mathcal{F}$ be the set of all f_i. The task is to distinguish between the following two distributions.*

1. *Null distribution $\mathcal{Q}$: $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, where $\mathbf{b} \in \Gamma^m$ is sampled uniformly at random.*

2. *Planted distribution $\mathcal{P}$: $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, where we sample $\mathbf{s} \in \Sigma^n$ at random and set*

$$\mathbf{b}_i = f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)}).$$

For a given pair $(\mathbf{H}, \mathcal{F})$, let $\mathcal{Q}_{\mathbf{H},\mathcal{F}}$ denote the distribution on vectors $\mathbf{b}$ as sampled in (1), and $\mathcal{P}_{\mathbf{H},\mathcal{F}}$ denote the distribution on vectors $\mathbf{b}$ as sampled in (2).

6.1 Low Complexity Embedding Attacks

As discussed in Section 1.1 of the introduction, there are known counterexamples to the security of Goldreich's PRG over arbitrary expanders [OST18]. In this subsection we explore and then immediately rule out the possibility of using these attacks to break Problem 1. We start by giving a hypothesis testing framework which *significantly generalizes* the set of attacks given by Oliveira, Santhanam, and Tell [OST18], and then show that *the entropy of the random functions alone is enough for Problem 1 to unconditionally resist all such attacks.*

The Attack on Goldreich's PRG. Oliveira, Santhanam, and Tell [OST18] consider the following decision problem,[15] which is similar to Problem 1 but uses *small alphabets* and a *single, low complexity* predicate. Given a $(1-o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix $\mathbf{H}$ and a predicate $f : \{0,1\}^k \to \{0,1\}$, distinguish between the following distributions:

1. $(\mathbf{H}, f, \mathbf{b})$, where $\mathbf{b} \in \{0,1\}^m$ is sampled at random.
2. $(\mathbf{H}, f, \mathbf{b})$, where we sample $\mathbf{s} \in \{0,1\}^n$ at random and set

$$\mathbf{b}_i = f(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)}).$$

All of their attacks now assume the same general format. They (non-constructively) prove the existence of an expanding $(n^{\log^{O(1)} n}, n, \log^{O(1)} n)$-matrix $\mathbf{H}$ such that the above problem can be solved in quasi-polynomial time, *assuming the predicate f is of a particular form*; both the matrix $\mathbf{H}$ and the predicate f must be representable using a sufficiently small $\mathrm{AC}^0[+]$ circuit. When these properties hold, they demonstrate that for every choice of seed $\mathbf{s}$, the vector $\mathbf{b}$ in distribution (2) will be the truth table of a small (unknown) $\mathrm{AC}^0[+]$ circuit. Their attacks then work by leveraging *natural property algorithms* (see e.g. [Raz87, Smo87, RR94, CIKK16]), which can distinguish with constant advantage between the truth table of any small $\mathrm{AC}^0[+]$ circuit and a random vector of the same length.

A different way to look at the attacks is as follows. Let $\mathcal{L} \subset \{0,1\}^m$ be the set of all truth tables for small $\mathrm{AC}^0[+]$ circuits. Use a natural property algorithm to determine if

$$\mathrm{dist}(\mathbf{b}, \mathcal{L}) = 0,$$

and choose to accept or reject based on the result. Recall that $\mathrm{dist}(\mathbf{b}, \mathcal{L})$ is the Hamming distance between $\mathbf{b}$ and the closest vector in $\mathcal{L}$, so $\mathrm{dist}(\mathbf{b}, \mathcal{L}) = 0$ if and only if $\mathbf{b} \in \mathcal{L}$.

Our Generalized Framework. Here we define the notion of a *low complexity embedding game*. As we discuss later, this game significantly generalizes the Oliveira-Santhanam-Tell attack. Before giving the game, we define the *generalized distance* between two vectors $\mathbf{u}, \mathbf{v} \in \{\mathbb{F} \cup \perp\}^m$ to be

$$\mathrm{dist'}(\mathbf{u}, \mathbf{v}) = \left|\left\{ i : \mathbf{u}_i \neq \mathbf{v}_i \text{ and } \mathbf{u}_i \neq \perp \text{ and } \mathbf{v}_i \neq \perp \right\}\right|,$$

i.e. the number of coordinates on which $\mathbf{u}, \mathbf{v}$ disagree but excluding those coordinates with a "don't care" symbol $\perp$. For a vector $\mathbf{u} \in \{\mathbb{F} \cup \perp\}^m$ and a set $\mathcal{L} \subseteq \{\mathbb{F} \cup \perp\}^m$, we define $\mathrm{dist'}(\mathbf{u}, \mathcal{L})$ to be the minimum of $\mathrm{dist'}(\mathbf{u}, \mathbf{v})$ over all $\mathbf{v} \in \mathcal{L}$.

Definition 6.1 (Low Complexity Embedding Game)**.** [16] *The game goes as follows.*

[15] The authors phrase this problem in terms of graphs and use slightly different terminology, but our version is equivalent.

[16] Technically speaking, the game allows for the adversary to choose more general mappings than just injective embeddings.

1. *The challenger fixes parameters m, n, k and alphabets Σ, Γ. She also fixes the description of*
   (a) *A null distribution $\mathcal{D}_{\mathbf{H},\mathcal{F}}$ over vectors $\mathbf{b} \in \Gamma^m$,*
   (b) *A planted distribution $\mathcal{D}'_{\mathbf{H},\mathcal{F}}$ over vectors $\mathbf{b} \in \Gamma^m$, and*
   (c) *A distribution $\mathcal{Z}$ over sets of functions $\mathcal{F} = \{f_i : \Sigma^k \to \Gamma\}_{i\in[m]}$.*

   Both $\mathcal{D}_{\mathbf{H},\mathcal{F}}$ and $\mathcal{D}'_{\mathbf{H},\mathcal{F}}$ depend on a $(1-o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix $\mathbf{H}$ to be fixed later by the adversary, and a set of functions $\mathcal{F} = \{f_i : \Sigma^k \to \Gamma\}$ to be sampled later by the challenger.
2. *After viewing the descriptions of $\mathcal{D}_{\mathbf{H},\mathcal{F}}$, $\mathcal{D}'_{\mathbf{H},\mathcal{F}}$, and $\mathcal{Z}$, the adversary spends unbounded time to choose a $(1 - o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix $\mathbf{H}$ along with a tuple $(\mathbb{F}, d, \mathcal{L}, \Psi)$, where*
   (a) *$\mathbb{F}$ is any finite field,*
   (b) *d is an embedding dimension parameter,*
   (c) *$\mathcal{L} \subseteq \{\mathbb{F} \cup \perp\}^{md}$ is any subset of vectors, and*
   (d) *$\Psi : \mathbb{F}^{md} \to \{\mathbb{F} \cup \perp\}^{md}$ is any transformation.*
3. *The challenger samples a set of functions $\mathcal{F}$ from the distribution $\mathcal{Z}$.*
4. *Based on $\mathbf{H}$, $\mathcal{F}$, the description of $\mathcal{D}_{\mathbf{H},\mathcal{F}}$, and the description of $\mathcal{D}'_{\mathbf{H},\mathcal{F}}$, the adversary spends unbounded time to fix a tuple (Φ, π, h), where*
   (a) *$\Phi = \{\phi_i : \Gamma \to \mathbb{F}^d\}_{i\in[m]}$ is any set of (not necessarily injective) functions.*
   (b) *$\pi : \mathbb{F}^{md} \to \mathbb{F}^{md}$ is any affine transformation.*
   (c) *$h : \{0, \ldots, md\} \to \{0, 1\}$ is an acceptance predicate.*

The adversary wins the game if and only if

$$\left| \Pr_{\mathbf{b}\sim\mathcal{D}'_{\mathbf{H},\mathcal{F}}} \left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1)\| \ldots \|\phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1\right] - \Pr_{\mathbf{b}\sim\mathcal{D}_{\mathbf{H},\mathcal{F}}} \left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1)\| \ldots \|\phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1\right] \right| = \Omega(1).$$

Notice that the adversary fixes some components of the test in advance and other components after gaining knowledge of the functions in $\mathcal{F}$. Intuitively, the first parameters $(\mathbb{F}, d, \mathcal{L}, \Psi)$ are generic, in the sense that they give the description of a "testing structure" which is meant to be compatible with the adversarially chosen expander $\mathbf{H}$.

The parameters (Φ, π, h) fixed after viewing $\mathcal{F}$ allow the adversary to perform a best-possible coordinate-wise mapping of an observed vector $\mathbf{b}$ into a vector over $\mathbb{F}^{md}$. The mappings in Φ are not required to be injective, which means that an adversary can choose to aggregate different symbols of the alphabet Γ in an optimal manner. Notice that, while π is "just" an affine transformation, it does have the power to arbitrarily permute coordinates, perform linear computations, and erase coordinates (by setting them equal to a designated constant which is later mapped by Ψ to $\perp$). This gives the adversary significant power to massage the vectors $\mathbf{b}$ into a form which is compatible with the "testing structure" defined by $(\mathbb{F}, d, \mathcal{L}, \Psi)$, especially when the alphabet Γ does not have a natural field structure (as is the case in Problem 1).

Keep in mind that at every step, we grant the adversary unbounded time to pick the best matrix $\mathbf{H}$, the best tuple $(\mathbb{F}, d, \mathcal{L}, \Psi)$, and the best tuple (Φ, π, h). We also place no requirements on the time required to compute

$$\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1)\| \ldots \|\phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big),$$

even though $\mathcal{L}$ is an arbitrary structure chosen by the adversary. As of today, we only know of efficient algorithms to perform this distance computation, even approximately, for very specific sets $\mathcal{L}$ (e.g. the set of truth tables for small $\mathrm{AC}^0[+]$ circuits).

Re-Framing the Oliveira-Santhanam-Tell Attack. We view the attack in terms of a low complexity embedding game:

1. *The challenger fixes parameters $(m, n, k) = (n^{\log^{O(1)} n}, n, \log^{O(1)} n)$ and alphabets $(\Sigma, \Gamma) = (\{0,1\}, \{0,1\})$. She also fixes*
   (a) *The null distribution $\mathcal{D}_{\mathbf{H},\mathcal{F}}$ as the uniform distribution over vectors $\mathbf{b} \in \{0,1\}^m$.*
   (b) *The planted distribution $\mathcal{D}'_{\mathbf{H},\mathcal{F}}$ as the distribution over vectors $\mathbf{b} \in \{0,1\}^m$ sampled by picking $\mathbf{s} \in \{0,1\}^n$ at random and then setting*
   $$\mathbf{b}_i = f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)}).$$
   (c) *Any distribution $\mathcal{Z}$ over sets of functions $\mathcal{F} = \{f_i : \Sigma^k \to \Gamma\}_{i\in[m]}$ such that for all $\mathcal{F}$ drawn from $\mathcal{Z}$:*
      i. *For all $i, j \in [m]$, we have $f_i = f_j$.*
      ii. *For all $i \in [m]$, the function f_i is computable by a sufficiently small $AC^0[+]$ circuit.*
2. *The adversary constructs a $(1 - o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix $\mathbf{H}$ that is described by a sufficiently small $AC^0[+]$ circuit. She also fixes a tuple $(\mathbb{F}, d, \mathcal{L}, \Psi)$, where*
   (a) *$\mathbb{F} := \mathbb{F}_2$,*
   (b) *$d := 1$,*
   (c) *$\mathcal{L} \subseteq \{\mathbb{F}_2 \cup \perp\}^{md}$ is the set of all truth tables for sufficiently small $AC^0[+]$ circuits (the symbol $\perp$ is not used).*
   (d) *$\Psi : \mathbb{F}_2^{md} \to \{\mathbb{F}_2 \cup \perp\}^{md}$ is the identity transformation (the symbol $\perp$ is again not used).*
3. *The challenger samples a set of functions $\mathcal{F}$ from the distribution $\mathcal{Z}$.*
4. *The adversary spends unbounded time to fix a tuple (Φ, π, h), where*
   (a) *Every $\phi_i \in \Phi$ just casts from $\{0,1\}$ to the field $\mathbb{F}_2$ by mapping zero to zero and one to one.*
   (b) *π is the identity transformation.*
   (c) *h is the function which outputs 1 iff its input is zero.*

Now because most functions were set to the identity, if we abuse notation and assume that $\mathbf{b}$ is already over the field $\mathbb{F}_2$ then

$$\left| \Pr_{\mathbf{b}\sim\mathcal{D}'_{\mathbf{H},\mathcal{F}}} \left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1)\| \ldots \|\phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1\right] - \Pr_{\mathbf{b}\sim\mathcal{D}_{\mathbf{H},\mathcal{F}}} \left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1)\| \ldots \|\phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1\right] \right|$$

is the same as

$$\left| \Pr_{\mathbf{b}\sim\mathcal{D}'_{\mathbf{H},\mathcal{F}}} \left[\text{dist'}\Big(\mathbf{b}, \mathcal{L}\Big)\Big) = 0\right] \right.$$
$$\left. - \Pr_{\mathbf{b}\sim\mathcal{D}_{\mathbf{H},\mathcal{F}}} \left[\text{dist'}\Big(\mathbf{b}, \mathcal{L}\Big)\Big) = 0\right] \right|.$$

The authors showed that this quantity is $\Omega(1)$, so the adversary wins the game.

Regarding the Adversary's Strength. Observe that, by allowing the adversary to fix *any* subset $\mathcal{L} \subseteq \{\mathbb{F} \cup \perp\}^{md}$, any problem where the set of functions $\mathcal{F}$ does not come from a high entropy distribution is immediately broken. As a simple example, consider the noisy kXOR problem with a *random* factor graph. Because the predicates f_i are all just addition over $\mathbb{F}_2$, and because the adversary has knowledge of the factor graph before fixing $\mathcal{L}$, she can just set $\mathcal{L}$ to be the set of all (noiseless) vectors $\mathbf{b}$ that could be sampled in the planted distribution. Now if dist'$(\mathbf{b}, \mathcal{L})$ is small, we know that the vector $\mathbf{b}$ was almost certainly drawn from the planted distribution, and if dist'$(\mathbf{b}, \mathcal{L})$ is large then we know that $\mathbf{b}$ was almost certainly drawn from the null (totally random) distribution.

In this way the adversary is unreasonably powerful. But for the purposes of proving lower bounds, this is only better; we rule out adversaries that capture the known attacks on Goldreich's PRG [OST18] as well as some attacks on noisy kXOR that are conjectured to be impossible to implement efficiently.

6.1.1 Our Lower Bound Against Low Complexity Embedding Attacks

To keep track of the parameters, recall Problem 1 (which is just the corruption-free version of the decision problem in Conjecture 3.1, the LARP-CSP Conjecture):

Problem 1 (Restated). *Let $\mathbf{H}$ be any $(1-o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix, where $k = (\log n)^{\Theta(1)}$ and $m \leq n^{o(k)}$. Let Σ, Γ be any alphabets satisfying $|\Sigma| = (nm)^{\log^{\Theta(1)}(nm)}$ and $|\Gamma| \leq |\Sigma|^{3k/4}$. Sample m random functions $f_i : \Sigma^k \to \Gamma$, and let $\mathcal{F}$ be the set of all f_i. The task is to distinguish between the following two distributions.*

1. *Null distribution $\mathcal{Q}$: $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, where $\mathbf{b} \in \Gamma^m$ is sampled uniformly at random.*
2. *Planted distribution $\mathcal{P}$: $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, where we sample $\mathbf{s} \in \Sigma^n$ at random and set*

$$\mathbf{b}_i = f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)}).$$

For a given pair $(\mathbf{H}, \mathcal{F})$, let $\mathcal{Q}_{\mathbf{H},\mathcal{F}}$ denote the distribution on vectors $\mathbf{b}$ as sampled in (1), and $\mathcal{P}_{\mathbf{H},\mathcal{F}}$ denote the distribution on vectors $\mathbf{b}$ as sampled in (2).

Now we give a formal statement of the lower bound.

Theorem 6.2. *Fix any $(1 - o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix $\mathbf{H}$, where $k = (\log n)^{\Theta(1)}$ and $m \leq n^{o(k)}$. Let Σ, Γ be any alphabets satisfying $|\Sigma| = (nm)^{\log^{\Theta(1)}(nm)}$ and $|\Gamma| \leq |\Sigma|^{3k/4}$. Also fix any*

1. *Finite field $\mathbb{F}$ of order $2 \leq |\mathbb{F}| \leq 2^{|\Sigma|^{o(k)}}$,*
2. *Embedding dimension parameter $1 \leq d \leq m^{O(1)}$,*
3. *Subset $\mathcal{L} \subseteq \{\mathbb{F} \cup \perp\}^{md}$, and*

4. *Transformation* $\Psi : \mathbb{F}^{md} \to \{\mathbb{F} \cup \perp\}^{md}$.

After this, sample the set of functions $\mathcal{F}$ *as in Problem 1. Then with probability* $1 - \exp\left\{-|\Sigma|^{\Omega(k)}\right\}$*, the following holds. For all*

1. *Functions* $\phi_1, \ldots, \phi_m : \Gamma \to \mathbb{F}^d$,
2. *Affine transformations* $\pi : \mathbb{F}^{md} \to \mathbb{F}^{md}$, *and*
3. *Acceptance predicates* $h : \{0, \ldots, md\} \to \{0, 1\}$,

we have

$$\left| \Pr_{\mathbf{b} \sim \mathcal{P}_{\mathbf{H},\mathcal{F}}} \left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1) \| \ldots \| \phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1 \right] \right.$$
$$\left. - \Pr_{\mathbf{b} \sim \mathcal{Q}_{\mathbf{H},\mathcal{F}}} \left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1) \| \ldots \| \phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1 \right] \right| \leq |\Sigma|^{-\Omega(k)},$$

where $\mathcal{Q}_{\mathbf{H},\mathcal{F}}$ *and* $\mathcal{P}_{\mathbf{H},\mathcal{F}}$ *are the null and planted distributions for Problem 1, respectively.*

In the proof Theorem 6.2, we need a way to formalize the dependencies between random variables.

Definition 6.3 (Dependency Graph)**.** *Let* $X_1, \ldots X_s$ *be random variables. The* dependency graph *for* $X_1, \ldots X_s$ *has one vertex for each random variable, and two variables* X_i, X_j *are connected by an edge if and only if they are dependent.*

We also use a concentration bound due to Janson.

Lemma 6.4 ([Jan04], basic version of Corollary 2.2)**.** *Let* $X = \sum_{i \in [r]} X_i$ *be the sum of* r *random variables* $X_1, \ldots, X_r$*, each distributed as Bernoulli variables. Let* G *be the dependency graph for* $X_1, \ldots, X_r$*. Then*

$$\Pr[X \geq \mathbb{E}[X] + t] \leq \exp\left\{-\Omega\left(\frac{t^2}{(\Delta(G)+1)r}\right)\right\}, \text{ and}$$

$$\Pr[X \leq \mathbb{E}[X] - t] \leq \exp\left\{-\Omega\left(\frac{t^2}{(\Delta(G)+1)r}\right)\right\},$$

where $\Delta(G)$ *is the maximum degree of* G.

Now we are ready to prove the lower bound.

Proof of Theorem 6.2. The proof will proceed in two parts:

1. Show that if $(\mathbb{F}, d, \mathcal{L}, \Psi)$ and (Φ, π, h) are fixed before sampling the random functions in $\mathcal{F}$, there is a sharp tail bound showing that the test almost surely has negligible advantage.
2. Union bound over all choices of (Φ, π, h), showing that with high probability there does not exist a good test even if an adversary is allowed to fix (Φ, π, h) after examining $\mathcal{F}$.

Claim 6.5. *Fix an (m, n, k)-graph $\mathbf{H}$ and any choice of $(\mathbb{F}, d, \mathcal{L}, \Psi)$ and (Φ, π, h). Then sample m random functions $f_i : \Sigma^k \to \Gamma$, and let $\mathcal{F}$ be the set of all f_i. With probability at least $1 - 2^{-|\Sigma|^{(4/5-o(1))k}}$ over the choice of $\mathcal{F}$,*

$$\left| \Pr_{\mathbf{b} \sim \mathcal{P}_{\mathbf{H},\mathcal{F}}} \left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1) \| \dots \| \phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1 \right] \right.$$
$$\left. - \Pr_{\mathbf{b} \sim \mathcal{Q}_{\mathbf{H},\mathcal{F}}} \left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1) \| \dots \| \phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1 \right] \right| < |\Sigma|^{-k/10}.$$

Proof. Let $\mathcal{U}$ be the distribution over sets of functions $\mathcal{F} = \{f_i : \Sigma^k \to \Gamma\}_{i \in [m]}$ where each of the functions f_i are sampled at random. Let T be shorthand for the test function

$$h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1) \| \dots \| \phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big),$$

which by definition of h always outputs a value in $\{0, 1\}$. It will suffice to show that

$$\Pr_{\mathcal{F} \sim \mathcal{U}} \left[\left| \Pr_{\mathbf{b} \sim \mathcal{P}_{\mathbf{H},\mathcal{F}}} [\mathsf{T}(\mathbf{b}) = 1] - \Pr_{\mathbf{b} \sim \mathcal{Q}_{\mathbf{H},\mathcal{F}}} [\mathsf{T}(\mathbf{b}) = 1] \right| \geq |\Sigma|^{-k/10} \right]$$

is at most $2^{-|\Sigma|^{(4/5-o(1))k}}$. We only prove the upper bound for

$$\Pr_{\mathcal{F} \sim \mathcal{U}} \left[\left(\Pr_{\mathbf{b} \sim \mathcal{P}_{\mathbf{H},\mathcal{F}}} [\mathsf{T}(\mathbf{b}) = 1] - \Pr_{\mathbf{b} \sim \mathcal{Q}_{\mathbf{H},\mathcal{F}}} [\mathsf{T}(\mathbf{b}) = 1] \right) \geq |\Sigma|^{-k/10} \right] \tag{1}$$

because the other case is nearly identical.

First consider the behavior under the null distribution. By definition of T there exists a set $\mathcal{S} \subseteq \Gamma^m$ (which does *not* depend on $\mathcal{F}$) such that $\mathsf{T}(\mathbf{b}) = 1$ if and only if $\mathbf{b} \in \mathcal{S}$. Since $\mathbf{b}$ is sampled at random in the null distribution, we have

$$\Pr_{\mathbf{b} \sim \mathcal{Q}_{\mathbf{H},\mathcal{F}}} [\mathsf{T}(\mathbf{b}) = 1] = \frac{|\mathcal{S}|}{|\Gamma|^m}$$

for all sets of functions $\mathcal{F}$. Thus (1) can be written as

$$\Pr_{\mathcal{F} \sim \mathcal{U}} \left[\left(\Pr_{\mathbf{b} \sim \mathcal{P}_{\mathbf{H},\mathcal{F}}} [\mathsf{T}(\mathbf{b}) = 1] - \frac{|\mathcal{S}|}{|\Gamma|^m} \right) \geq |\Sigma|^{-k/10} \right]. \tag{2}$$

Now consider the behavior under the planted distribution. Recall that we sample $\mathbf{b} \in \Gamma^m$ by first sampling $\mathbf{s} \in \Sigma^n$ uniformly at random, and then setting $\mathbf{b}_i = f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \dots, \mathbf{s}_{N_{\mathbf{H}}(i,k)})$. Let $\mathcal{P}_{\mathbf{H},\mathcal{F}}(\mathbf{s})$ be the function which outputs this vector $\mathbf{b}$ for a given vector $\mathbf{s}$. Because T always outputs a value in $\{0, 1\}$ (which we interpret as a real number), and because $\mathbf{s} \in \Sigma^n$ is sampled uniformly at random, (2) can be written as

$$\Pr_{\mathcal{F} \sim \mathcal{U}} \left[\left(\frac{1}{|\Sigma|^n} \sum_{\mathbf{s} \in \Sigma^n} \mathsf{T}(\mathcal{P}_{\mathbf{H},\mathcal{F}}(\mathbf{s})) - \frac{|\mathcal{S}|}{|\Gamma|^m} \right) \geq |\Sigma|^{-k/10} \right],$$

which is equivalent to

$$\Pr_{\mathcal{F} \sim \mathcal{U}} \left[\left(\sum_{\mathbf{s} \in \Sigma^n} \mathsf{T}(\mathcal{P}_{\mathbf{H},\mathcal{F}}(\mathbf{s})) - \frac{|\mathcal{S}||\Sigma|^n}{|\Gamma|^m} \right) \geq |\Sigma|^{-k/10} |\Sigma|^n \right]. \tag{3}$$

Let $\{X_{\mathbf{s}}\}_{\mathbf{s}\in\Sigma^n}$ be the set of 0/1 valued random variables defined as $X_{\mathbf{s}} = \mathsf{T}(\mathcal{P}_{\mathbf{H},\mathcal{F}}(\mathbf{s}))$, and denote their sum as $X = \sum_{\mathbf{s}\in\Sigma^n} X_{\mathbf{s}}$. By the randomness of the functions in $\mathcal{F}$, for all $\mathbf{s} \in \Sigma^n$ we have

$$\mathbb{E}_{\mathcal{F}\sim\mathcal{U}}[X_{\mathbf{s}}] = \frac{|\mathcal{S}|}{|\Gamma|^m},$$

which implies that

$$\mathbb{E}_{\mathcal{F}\sim\mathcal{U}}[X] = \frac{|\mathcal{S}||\Sigma|^n}{|\Gamma|^m}.$$

Therefore (3) is equivalent to

$$\Pr_{\mathcal{F}\sim\mathcal{U}}\left[X \geq \mathbb{E}_{\mathcal{F}\sim\mathcal{U}}[X] + |\Sigma|^{-k/10}|\Sigma|^n\right].$$

From here our plan is give a concentration bound. Observe that $X_{\mathbf{s}}$ and $X_{\mathbf{s}'}$ are independent if and only if, for all $i \in [m]$, we have $(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)}) \neq (\mathbf{s}'_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}'_{N_{\mathbf{H}}(i,k)})$. This is because the evaluation of each function f_i is independently random on each of its inputs. So the dependency graph G for the random variables $X_{\mathbf{s}}$ satisfies $\Delta(G) \leq \frac{m|\Sigma|^n}{|\Sigma|^k}$. Applying Lemma 6.4, we get

$$\begin{aligned}\Pr_{\mathcal{F}\sim\mathcal{U}}\left[X \geq \mathbb{E}_{\mathcal{F}\sim\mathcal{U}}[X] + |\Sigma|^{-k/10}|\Sigma|^n\right] \leq& 2^{-\Omega\left(\frac{|\Sigma|^{2n}|\Sigma|^{-k/5}}{|\Sigma|^{2n}|\Sigma|^{-k}m}\right)} \\ \leq& 2^{-\Omega\left(|\Gamma|^{4k/5}/m\right)}\end{aligned}$$

Because $|\Sigma| = m^{\omega(1)}$, this probability is upper bounded as $2^{-|\Sigma|^{(4/5-o(1))k}}$. □

All that's left is to union bound over the choices for (Φ, π, h). Using that $2 \leq |\mathbb{F}| \leq 2^{|\Gamma|^{o(1)}}$, $d \leq m^{O(1)}$, $|\Gamma| \leq |\Sigma|^{3k/4}$, and $|\Sigma| = m^{\omega(1)}$, we have

1. $(|\mathbb{F}|^d)^{m|\Gamma|} \leq 2^{|\Sigma|^{(3/4+o(1))k}}$ choices for $\Phi = \{\phi_i : \Gamma \to \mathbb{F}^d\}_{i\in[m]}$.
2. $|\mathbb{F}|^{O((md)^2)} \leq 2^{|\Sigma|^{o(k)}}$ choices for $\pi : \mathbb{F}^{md} \to \mathbb{F}^{md}$.
3. $2^{md+1} = 2^{|\Sigma|^{o(1)}}$ choices for $h : \{0, 1, \ldots, md\} \to \{0, 1\}$.

So the total number of choices for (Φ, π, h) is $2^{|\Sigma|^{(3/4+o(1))k}}$. Using Claim 6.5 we know that for a fixed choice of (Φ, π, h), with probability at least $1 - 2^{-|\Sigma|^{(4/5-o(1))k}}$ over the choice of $\mathcal{F}$,

$$\begin{aligned}\Bigg|&\Pr_{\mathbf{b}\sim\mathcal{P}_{\mathbf{H},\mathcal{F}}}\left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1)\|\ldots\|\phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1\right] \\ &- \Pr_{\mathbf{b}\sim\mathcal{Q}_{\mathbf{H},\mathcal{F}}}\left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1)\|\ldots\|\phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1\right]\Bigg| < |\Sigma|^{-k/10}.\end{aligned}$$

Thus with probability at least

$$1 - 2^{-|\Sigma|^{(4/5-o(1))k}} \cdot 2^{|\Sigma|^{(3/4+o(1))k}} > 1 - 2^{-|\Sigma|^{\Omega(k)}}$$

over the choice of $\mathcal{F}$, every choice of (Φ, π, h) satisfies

$$\left| \Pr_{\mathbf{b}\sim\mathcal{P}_{\mathbf{H},\mathcal{F}}} \left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1)\| \dots \|\phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1\right] \right.$$
$$\left. - \Pr_{\mathbf{b}\sim\mathcal{Q}_{\mathbf{H},\mathcal{F}}} \left[h\Big(\text{dist'}\Big(\Psi\big(\pi\big(\phi(\mathbf{b}_1)\| \dots \|\phi(\mathbf{b}_m)\big)\big), \mathcal{L}\Big)\Big) = 1\right] \right| < |\Sigma|^{-k/10}.$$

□

6.2 Low Degree Polynomial Algorithms

In this sub-section we rule out low degree polynomial algorithms for Problem 1, up to a nearly optimal degree cutoff. As in Section 6.1, such a lower bound is possible because of the massive entropy coming from the random functions in $\mathcal{F}$; the *signal-to-noise ratio* is very small.

Input Formulation. Recall Problem 1 (which is just the corruption-free version of the decision problem in Conjecture 3.1, the LARP-CSP Conjecture):

Problem 1 (Restated). *Let $\mathbf{H}$ be any $(1-o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix, where $k = (\log n)^{\Theta(1)}$ and $m \le n^{o(k)}$. Let Σ, Γ be any alphabets satisfying $|\Sigma| = (nm)^{\log^{\Theta(1)}(nm)}$ and $|\Gamma| \le |\Sigma|^{3k/4}$. Sample m random functions $f_i : \Sigma^k \to \Gamma$, and let $\mathcal{F}$ be the set of all f_i. The task is to distinguish between the following two distributions.*

1. *Null distribution $\mathcal{Q}$: $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, where $\mathbf{b} \in \Gamma^m$ is sampled uniformly at random.*
2. *Planted distribution $\mathcal{P}$: $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, where we sample $\mathbf{s} \in \Sigma^n$ at random and set*

$$\mathbf{b}_i = f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \dots, \mathbf{s}_{N_{\mathbf{H}}(i,k)}).$$

To analyze this problem in terms of low degree polynomials, we need to find the right input formulation. Indeed if we only examine the pair $(\mathbf{H}, \mathbf{b})$ then $\mathbf{b}$ is statistically random, because each f_i is a random function. We will instead represent the problem using a carefully chosen representation of the set of all tuples $(\sigma_1, \dots, \sigma_k) \in \Sigma^k$ such that there exists an $i \in [m]$ with $f_i(\sigma_1, \dots, \sigma_k) = \mathbf{b}_i$. This succinctly combines the information contained in $\mathcal{F}$ and $\mathbf{b}$, and it aligns with the fact that Problem 1 can be viewed equivalently as a planted hypergraph problem. Concretely speaking, via the randomness of the functions f_i, Problem 1 is equivalent to the following:

Problem 2. *Let $\mathbf{H}$ be any $(1 - o(1), n^{1-o(1)})$-expanding (m, n, k)-matrix, where $k = (\log n)^{\Theta(1)}$ and $m \le n^{o(k)}$. Let Σ, Γ be any alphabets satisfying $|\Sigma| = (nm)^{\log^{\Theta(1)}(nm)}$ and $|\Gamma| \le |\Sigma|^{3k/4}$. Define*

$$\mathcal{X} := \{(j_1, \dots, j_k) : \exists i \in [m] \text{ s.t. the nonzero entries of } \mathbf{H}_i \text{ are in columns } j_1, \dots, j_k\}.$$

Sample a hypergraph $K = (V, E)$ with vertex set $V = [n] \times \Sigma$, where the edge set E is sampled in one of two ways:

1. *Null distribution $\mathcal{Q}$: For all $(j_1, \dots, j_k) \in \mathcal{X}$, for all $(\sigma_{j_1}, \dots, \sigma_{j_k}) \in \Sigma^k$, add the edge $((j_1, \sigma_{j_1}), \dots, (j_k, \sigma_{j_k}))$ independently with probability $1/|\Gamma|$.*
2. *Planted distribution $\mathcal{P}$: Perform the same procedure as in distribution $\mathcal{Q}$. Then sample $\mathbf{s} \in \Sigma^k$ at random, and for all $(j_1, \dots, j_k) \in \mathcal{X}$ add the edge $((j_1, \mathbf{s}_{j_1}), \dots, (j_k, \mathbf{s}_{j_k}))$ if it doesn't already exist.*

The task is to determine, given $(\mathbf{H}, K)$*, which distribution* K *was sampled from.*

Remark 6.6. The pair $(\mathbf{H}, K)$ contains significantly less information than the tuple $(\mathbf{H}, \mathcal{F}, \mathbf{b})$, because in Problem 1 we are given the entire truth table for each function f_i whereas in this problem we only learn, for all $i \in [m]$, the set of all $(\sigma_{j_1}, \ldots, \sigma_{j_k})$ such that $f_i(\sigma_{j_1}, \ldots, \sigma_{j_k}) = \mathbf{b}_i$. But all of the lost information is "irrelevant," in the sense that we can efficiently solve Problem 2 using an oracle for Problem 1 by just re-sampling the missing truth table entries from an appropriate distribution to simulate the tuple $(\mathbf{H}, \mathcal{F}, \mathbf{b})$.

Interpretation as a Vector Over the Reals. We assume that our low degree polynomial algorithms take as input the *real-valued indicator vector* for $(\mathbf{H}, K)$. In particular, $\mathbf{H}$ is represented over the reals by mapping 0 to 0 and 1 to 1, and K is represented as vector with one coordinate for every tuple $((j_1, \mathbf{s}_{j_1}), \ldots, (j_k, \mathbf{s}_{j_k}))$, where the coordinate is set to 0 if the corresponding hyperedge is not present in K and 1 if the corresponding hyperedge is present in K. This is the natural formulation used to prove lower bounds for planted graph and hypergraph problems [MVW24, DMW25].

Low Degree Polynomial Algorithms. Consider an arbitrary hypothesis testing problem, with a null distribution $\mathcal{Q}$ over real vectors Y and a planted distribution $\mathcal{P}$ over real vectors Y of the same length. As discussed in [Hop18, KWB19] a standard way to formalize the distinguishing power of a polynomial g is to examine the ratio

$$\frac{\mathbb{E}_{Y\sim\mathcal{P}}[g(Y)] - \mathbb{E}_{Y\sim\mathcal{Q}}[g(Y)]}{\sqrt{\mathrm{Var}_{Y\sim\mathcal{Q}}[g(Y)]}}. \tag{4}$$

If this ratio is large, then $g(Y)$ will typically be much larger under the planted distribution than under the null distribution, even after accounting for the random fluctuations in the output of $g(Y)$ under the null distribution. The *degree-d advantage*[17] is defined as

$$\max_{g \text{ is a degree } \leq d \text{ polynomial}} \frac{\mathbb{E}_{Y\sim\mathcal{P}}[g(Y)] - \mathbb{E}_{Y\sim\mathcal{Q}}[g(Y)]}{\sqrt{\mathrm{Var}_{Y\sim\mathcal{Q}}[g(Y)]}}.$$

We say that degree-$\leq d$ polynomials fail to distinguish between $\mathcal{P}$ and $\mathcal{Q}$ when the degree-d advantage is $o(1)$ [Hop18, KWB19]. In many ways degree-$\leq d$ polynomials are a proxy for "combinatorial" algorithms that run in time $n^{d/\log^{O(1)} n}$ [KWB19], so if degree-$\leq d$ polynomials are ineffective for the hypothesis testing problem this suggests that $n^{d/\log^{O(1)} n}$ time "combinatorial" algorithms are also ineffective, where n is the number of coordinates in Y.

Now we specialize these ideas for Problem 2. Observe that $\mathbf{H}$ is the same in both the null and planted distributions, so we can assume without loss of generality that our polynomial does not depend on $\mathbf{H}$. We know that all tuples $((j_1, \sigma_{j_1}), \ldots, (j_k, \sigma_{j_k}))$ such that $(j_1, \ldots, j_k) \notin \mathcal{X}$ will never appear as an edge in K, so we can also assume without loss of generality that our polynomial does not depend on these. In light of this, define Y as the length $m' = m|\Sigma|^k$ *real-valued indicator vector* for the remaining possible hyperedges, i.e. the tuples $((j_1, \sigma_{j_1}), \ldots, (j_k, \sigma_{j_k}))$ such that $(j_1, \ldots, j_k) \in \mathcal{X}$. We have one coordinate in Y for each of these tuples; the coordinate is set to 0 if the corresponding hyperedge is not present in K and 1 if the corresponding hyperedge is present in K.

We may assume without loss of generality that $\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y)] = 0$, because if $\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y)] \neq 0$ then the polynomial $g' = g - \mathbb{E}_{Y\sim\mathcal{Q}}[g(Y)]$ has the same ratio (4) and degree, but satisfies $\mathbb{E}_{Y\sim\mathcal{Q}}[g'(Y)] = 0$. Thus we can rewrite (4) as

$$\frac{\mathbb{E}_{Y\sim\mathcal{P}}[g(Y)]}{\sqrt{\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y)^2]}}.$$

[17] In some works this is referred to as the "norm of the degree-d likelihood ratio."

Because every coordinate of Y, even after normalization (see below), will either equal x or y, we can also assume without loss of generality that g is multilinear. This is because every factor $Y^x_{((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))}$ with $x \neq 1$ can be replaced with a linear function $(aY_{((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))} + b)$, where a and b are chosen so that the linear function and the original factor agree when $Y_{((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))}$ is either x or y.

All of this is to say that, if we want to show that the degree-d advantage for Problem 2 is $o(1)$, it will suffice to prove that

$$\max \frac{\mathbb{E}_{Y\sim\mathcal{P}}[g(Y)]}{\sqrt{\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y)^2]}} = o(1),$$

where the optimization ranges over all degree-$\leq d$ multilinear polynomials g such that $\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y)] = 0$.

Normalization. To simplify the behavior of $g(Y)$ under the null distribution $\mathcal{Q}$, we define a *normalization mapping* $\phi : 0, 1 \to \mathbb{R}$ that will be applied to each coordinate of Y. Recall that under $\mathcal{Q}$, every coordinate of Y is sampled independently from $\mathrm{Ber}(|\Gamma|^{-1})$. Setting $p := |\Gamma|^{-1}$, we define ϕ as $\phi(0) := -\sqrt{\frac{p}{1-p}}$ and $\phi(1) := \sqrt{\frac{1-p}{p}}$. This gives, for all coordinates $((j_1,\sigma_1),\dots,(j_k,\sigma_k))$,

$$\mathbb{E}_{Y\sim\mathcal{Q}}[\phi(Y_{((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))})] = -\sqrt{\frac{p}{1-p}} \cdot (1-p) + \sqrt{\frac{1-p}{p}} \cdot p = 0 \tag{5}$$

and

$$\mathbb{E}_{Y\sim\mathcal{Q}}[\phi(Y_{((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))})^2] = \frac{p}{1-p} \cdot (1-p) + \frac{1-p}{p} \cdot p = p + (1-p) = 1. \tag{6}$$

Let $Y^{(\phi)}$ be shorthand for the vector obtained by applying ϕ to each coordinate of Y. Normalization is a linear mapping, so we can assume without loss of generality that our polynomials g take as input $Y^{(\phi)}$; any degree d polynomial with input Y can be converted into a degree d polynomial with input $Y^{(\phi)}$ and identical output distribution.

Recall that Y will be of length $m' = m|\Sigma|^k$. We make use of the following simple but powerful lemma.

Lemma 6.7 (Based on [Hop18, KWB19], specialized to Problem 2)**.** *Let $\mathcal{Q}$ and $\mathcal{P}$ be the null and planted distributions, respectively, from Problem 2, and let ϕ be the normalization mapping defined as $\phi(0) := -\sqrt{\frac{|\Gamma|^{-1}}{1-|\Gamma|^{-1}}}$ and $\phi(1) := \sqrt{\frac{1-|\Gamma|^{-1}}{|\Gamma|^{-1}}}$. For any degree cutoff $d \leq m'$, let $\mathbf{c}$ be the length-$\left(\binom{m'}{\leq d} - 1\right)$ vector of expected values, under the distribution $\mathcal{P}$, for all multilinear monomials of degree at least 1 and at most d in the coordinates of $Y^{(\phi)}$. If*

$$\|\mathbf{c}\|_2 = o(1),$$

then the degree-d advantage for Problem 2 is $o(1)$.

Proof. As established previously, it will suffice to show that $\|\mathbf{c}\|_2 = o(1)$ implies

$$\max \frac{\mathbb{E}_{Y\sim\mathcal{P}}[g(Y^{(\phi)})]}{\sqrt{\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y^{(\phi)})^2]}} = o(1),$$

where the optimization ranges over all degree-$\leq d$ multilinear polynomials g such that $\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y^{(\phi)})] = 0$.

Let $\hat{\mathbf{g}}$ be the length-$\sum_{i=1}^{d}\binom{m'}{i}$ vector of coefficients for a degree-$\leq d$ multilinear polynomial g in the coordinates of $Y^{(\phi)}$. By normalization, we know that the expected value for each monomial under $\mathcal{Q}$ is zero, so if $\mathbb{E}_{T\sim\mathcal{P}}[g(T^{(\phi)})] = 0$ then the constant term is zero. As such we omit the constant term from $\hat{\mathbf{g}}$.

By definition of $\hat{\mathbf{g}}$ and $\mathbf{c}$, we have $\mathbb{E}_{Y\sim\mathcal{P}}[g(Y^{(\phi)})] = \langle\hat{\mathbf{g}},\mathbf{c}\rangle$, where $\langle\cdot,\cdot\rangle$ is the standard inner product. Now we analyze $\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y^{(\phi)})^2]$. For a multilinear monomial $M\in\mathcal{M}(m',d)$, we know by normalization that $\mathbb{E}_{Y\sim\mathcal{Q}}[M(Y^{(\phi)})^2]=1$. For two monomials $M\neq M'$, we know that $\mathbb{E}_{Y\sim\mathcal{Q}}[M(Y^{(\phi)})M'(Y^{(\phi)})]=0$, because the product must depend linearly on at least one coordinate of $Y^{(\phi)}$, and this normalized coordinate has an expected value of zero independently of all other coordinates. Thus if we index $\hat{\mathbf{g}}$ by the non-constant multilinear monomials $M\in\mathcal{M}(m',d)$,

$$\begin{aligned}\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y)^2] &= \sum_{M,M'\in\mathcal{M}(m',d)\text{ such that }M,M'\neq 1}\hat{\mathbf{g}}_M\hat{\mathbf{g}}_{M'}\mathbb{E}_{Y\sim\mathcal{Q}}[M(Y^{(\phi)})M'(Y^{(\phi)})]\\ &= \sum_{M\in\mathcal{M}(m',d)\text{ such that }M\neq 1}\hat{\mathbf{g}}_M^2\\ &=\langle\hat{\mathbf{g}},\hat{\mathbf{g}}\rangle.\end{aligned}$$

This gives

$$\max\frac{\mathbb{E}_{Y\sim\mathcal{P}}[g(Y^{(\phi)})]}{\sqrt{\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y^{(\phi)})^2]}} = \max\frac{\langle\hat{\mathbf{g}},\mathbf{c}\rangle}{\sqrt{\langle\hat{\mathbf{g}},\hat{\mathbf{g}}\rangle}}.$$

This is maximized when $\hat{\mathbf{g}}$ is a positive scalar multiple of $\mathbf{c}$, in which case the above quantity is $\|\mathbf{c}\|_2$. So if $\|\mathbf{c}\|_2=o(1)$ then the degree-d advantage is $o(1)$. □

6.2.1 The Low Degree Polynomial Lower Bound

Our lower bound may be stated as follows. It's nearly tight, as there exists a trivial degree-$n\log^{O(1)}n$ algorithm that solves Problem 2.

Theorem 6.8. *The degree-$n^{0.99}$ advantage for Problem 2 is $o(1)$. In other words, for all degree-$\leq n^{0.99}$ polynomials g,*

$$\frac{\mathbb{E}_{Y\sim\mathcal{P}}[g(Y)]-\mathbb{E}_{Y\sim\mathcal{Q}}[g(Y)]}{\sqrt{\mathrm{Var}_{Y\sim\mathcal{Q}}[g(Y)]}}=o(1).$$

Proof. By Lemma 6.7, it will suffice to show that $\|\mathbf{c}\|_2=o(1)$, where $\mathbf{c}$ is the length-$\left(\binom{m'}{\leq n^{0.99}}-1\right)$ vector of expected values, under the distribution $\mathcal{P}$, for all multilinear monomials of degree at least 1 and at most $\leq n^{0.99}$ in the coordinates of $Y^{(\phi)}$. This is equivalent to showing that $\|\mathbf{c}\|_2^2=o(1)$, or in other words

$$\sum_{M\in\mathcal{M}(m',n^{0.99})\text{ such that }M\neq 1}\mathbb{E}_{Y\sim\mathcal{P}}[M(Y^{(\phi)})]^2. \tag{7}$$

is bounded as $o(1)$.

We begin by writing the expected value for each monomial in terms of the probability that the planted edges fall entirely within the monomial.

Claim 6.9. *Let $M\in\mathcal{M}(m',n^{0.99})$ be any non-constant multilinear monomial, and write M as*

$$\Pi_{((j_1,\sigma_{j_1}),\ldots,(j_k,\sigma_{j_k}))\in\mathcal{S}}Y^{(\phi)}_{((j_1,\sigma_{j_1}),\ldots,(j_k,\sigma_{j_k}))},$$

where $\mathcal{S}\subset([n]\times\Sigma)^k$ is a subset of the edges represented by $Y^{(\phi)}$. Then

$$\mathbb{E}_{Y\sim\mathcal{P}}[M(Y^{(\phi)})]\leq|\Gamma|^{|\mathcal{S}|/2}\cdot\Pr_{Y\sim\mathcal{P}}\left[\sigma_{j_\ell}=\mathbf{s}_{j_\ell}\text{ for all }\ell\in[k]\text{ and }((j_1,\sigma_{j_1}),\ldots,(j_k,\sigma_{j_k}))\in\mathcal{S}\right].$$

Proof. We have that

$$\begin{aligned}
&\mathbb{E}_{Y\sim\mathcal{P}}[M(Y^{(\phi)})] = \\
&\quad \mathbb{E}_{Y\sim\mathcal{P}}\Big[M(Y^{(\phi)})\Big| \text{ there exists } \ell\in[k] \text{ and } ((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S} \text{ such that } \sigma_{j_\ell}\neq \mathbf{s}_{j_\ell}\Big] \\
&\qquad \cdot \Pr_{Y\sim\mathcal{P}}\Big[\text{ there exists } \ell\in[k] \text{ and } ((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S} \text{ such that } \sigma_{j_\ell}\neq \mathbf{s}_{j_\ell}\Big] \\
&\quad + \mathbb{E}_{Y\sim\mathcal{P}}\Big[M(Y^{(\phi)})\big|\sigma_{j_\ell}=\mathbf{s}_{j_\ell} \text{ for all } \ell\in[k] \text{ and } ((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S}\Big] \\
&\qquad \cdot \Pr_{Y\sim\mathcal{P}}\Big[\sigma_{j_\ell}=\mathbf{s}_{j_\ell} \text{ for all } \ell\in[k] \text{ and } ((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S}\Big].
\end{aligned}$$

For the first term, we condition on at least one edge in the monomial being outside of the planted set of edges. By Equation (5), the expected value for the variable corresponding to this edge is zero, independently of the other variables. So the first term is zero.

For the second term, we condition on all edges in the monomial being inside of the planted set of edges. Therefore every variable in the monomial will equal $\phi(1)=\sqrt{\frac{1-|\Gamma|^{-1}}{|\Gamma|}}$. This gives

$$\begin{aligned}
\mathbb{E}_{Y\sim\mathcal{P}}\Big[M(Y^{(\phi)})\big|\sigma_{j_\ell}=\mathbf{s}_{j_\ell} \text{ for all } \ell\in[k] \text{ and } ((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S}\Big] =& (\phi(1))^{|\mathcal{S}|} \\
=& \sqrt{\frac{1-|\Gamma|^{-1}}{|\Gamma|^{-1}}}^{|\mathcal{S}|} \\
\leq& \sqrt{|\Gamma|}^{|\mathcal{S}|}
\end{aligned}$$

Multiplying this by $\Pr_{Y\sim\mathcal{P}}\Big[\sigma_{j_\ell}=\mathbf{s}_{j_\ell} \text{ for all } \ell\in[k] \text{ and } ((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S}\Big]$ completes the proof. □

Now we bound the probability that all edges in a given monomial are inside of the planted set of edges. Here we appeal to the expansion of $\mathbf{H}$.

Claim 6.10. *Let $M\in\mathcal{M}(m',n^{0.99})$ be any non-constant multilinear monomial, and write M as*

$$\Pi_{((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S}} Y^{(\phi)}_{((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))},$$

where $\mathcal{S}\subset([n]\times\Sigma)^k$ is a subset of the hyperedges represented by $Y^{(\phi)}$. Then

$$\Pr_{Y\sim\mathcal{P}}\Big[\sigma_{j_\ell}=\mathbf{s}_{j_\ell} \textit{ for all } \ell\in[k] \textit{ and } ((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S}\Big] \leq |\Sigma|^{-(1-o(1))k|\mathcal{S}|}.$$

Proof. First we argue that if there exist two distinct $((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k})),((j'_1,\sigma'_{j'_1}),\dots,(j'_k,\sigma'_{j'_k}))\in\mathcal{S}$ such that $j_\ell=j'_\ell$ for all $\ell\in[k]$, then

$$\Pr_{Y\sim\mathcal{P}}\Big[\sigma_{j_\ell}=\mathbf{s}_{j_\ell} \text{ for all } \ell\in[k] \text{ and } ((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S}\Big] = 0 \leq |\Sigma|^{-(1-o(1))k|\mathcal{S}|}.$$

This is because, by the distinctness assumption, there must exist an $\ell\in[k]$ such that $\sigma_{j_\ell}\neq\sigma'_{j'_\ell}$. But because $j_\ell=j'_\ell$, the event in the probability statement only occurs if $\mathbf{s}_{j_\ell}=\sigma_{j_\ell}$ and $\mathbf{s}_{j_\ell}=\sigma'_{j_\ell}$ and thus $\mathbf{s}_{j_\ell}\neq\mathbf{s}_{j_\ell}$, which is not possible.

So assume that every tuple $((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S}$ has a distinct sub-tuple $(j_1,\dots j_k)$. Now because $\mathbf{H}$ is a $(1-o(1),n^{1-o(1)})$-expander, and $|\mathcal{S}|\leq n^{0.99}<n^{1-o(1)}$ by assumption, the set

$$\mathcal{I}=\{j:\text{ there exists } \ell\in[k] \text{ and } ((j_1,\sigma_{j_1}),\dots,(j_k,\sigma_{j_k}))\in\mathcal{S} \text{ such that } j=j_\ell\}$$

has cardinality at least $(1-o(1))k|\mathcal{S}|$. Over the randomness of $\mathbf{s} \in \Sigma^n$, this implies

$$\Pr_{Y\sim\mathcal{P}}\left[\mathbf{s}_{j_\ell} = \sigma_{j_\ell} \text{ for all } \ell \in [k] \text{ and } ((j_1,\sigma_{j_1}),\ldots,(j_k,\sigma_{j_k})) \in \mathcal{S}\right] = (1/|\Sigma|)^{(1-o(1))k|\mathcal{S}|}.$$

□

Now all that's left is to compute the sum of squared expectations over all monomials. Start with (7):

$$\sum_{M\in\mathcal{M}(m',n^{0.99}) \text{ such that } M\neq 1} \mathbb{E}_{Y\sim\mathcal{P}}[M(Y^{(\phi)})]^2$$

and then apply Claim 6.9 (where $\mathcal{S}_M$ is the set of variables in monomial M):

$$= \sum_{M\in\mathcal{M}(m',n^{0.99}) \text{ such that } M\neq 1} \left(|\Gamma|^{|\mathcal{S}_M|/2}\cdot \Pr_{Y\sim\mathcal{P}}\left[\sigma_{j_\ell} = \mathbf{s}_{j_\ell} \text{ for all } \ell\in[k] \text{ and } ((j_1,\sigma_{j_1}),\ldots,(j_k,\sigma_{j_k}))\in\mathcal{S}_M\right]\right)^2.$$

By Claim 6.10 this becomes

$$= \sum_{M\in\mathcal{M}(m',n^{0.99}) \text{ such that } M\neq 1} \left(|\Gamma|^{|\mathcal{S}_M|/2}\cdot|\Sigma|^{-(1-o(1))k|\mathcal{S}_M|}\right)^2.$$

Each of the terms only depends on the size of the (multilinear) monomial. Because there are $m' = m|\Sigma|^k$ variables in Y, we can rewrite the above as

$$\begin{aligned}
&= \sum_{1\le t\le n^{0.99}} \binom{m|\Sigma|^k}{t}\left(|\Gamma|^{t/2}\cdot|\Sigma|^{-(1-o(1))kt}\right)^2 \\
&\le \sum_{1\le t\le n^{0.99}} \left(m|\Sigma|^k\right)^t\left(|\Gamma|^{t/2}\cdot|\Sigma|^{-(1-o(1))kt}\right)^2 \\
&\le \sum_{1\le t\le n^{0.99}} \left(m|\Sigma|^k\right)^t\left(|\Sigma|^{3kt/8}\cdot|\Sigma|^{-(1-o(1))kt}\right)^2 && (|\Gamma|\le|\Sigma|^{3k/4}) \\
&\le \sum_{1\le t\le n^{0.99}} |\Sigma|^{(1+o(1))kt}|\Sigma|^{-((5/4)-o(1))kt} && (|\Sigma| = m^{\omega(1)}) \\
&\le \sum_{1\le t\le n^{0.99}} |\Sigma|^{-((1/4)+o(1))kt} \\
&\le |\Sigma|^{-\Omega(k)}.
\end{aligned}$$

Putting everything together, the degree-$n^{0.99}$ advantage for Problem 2 is $o(1)$. □

Remark 6.11. The only place in the proof where we use the expansion of $\mathbf{H}$ is in Claim 6.10. As such, the same lower bound proof actually works for polynomials of degree all the way up to the expansion cutoff.

6.3 Polynomial Calculus Refutations

In this section we prove lower bounds for Problem 1 against the *refutation* version of the polynomial calculus proof system. Recall that in this setting, the adversary is given a tuple $(\mathbf{H},\mathcal{F},\mathbf{b})$, and the goal is to certify that there does not exist a secret vector $\mathbf{s}\in\Sigma^n$ such that $\mathbf{b}_i = f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)},\ldots,\mathbf{s}_{N_{\mathbf{H}}(i,k)})$ for all $i\in[m]$. Our lower bounds are similar to those given by Applebaum and Lovett [AL16] for Goldreich's PRG.

To get started, recall Problem 1 (which as explained before is just the corruption-free version of the decision problem in Conjecture 3.1, the LARP-CSP Conjecture):

Problem 1 (Restated). *Let* $\mathbf{H}$ *be any* $(1-o(1), n^{1-o(1)})$*-expanding* (m, n, k)*-matrix, where* $k = (\log n)^{\Theta(1)}$ *and* $m \leq n^{o(k)}$*. Let* Σ, Γ *be any alphabets satisfying* $|\Sigma| = (nm)^{\log^{\Theta(1)}(nm)}$ *and* $|\Gamma| \leq |\Sigma|^{3k/4}$*. Sample* m *random functions* $f_i : \Sigma^k \to \Gamma$*, and let* $\mathcal{F}$ *be the set of all* f_i*. The task is to distinguish between the following two distributions.*

1. *Null distribution* $\mathcal{Q}$*:* $(\mathbf{H}, \mathcal{F}, \mathbf{b})$*, where* $\mathbf{b} \in \Gamma^m$ *is sampled uniformly at random.*
2. *Planted distribution* $\mathcal{P}$*:* $(\mathbf{H}, \mathcal{F}, \mathbf{b})$*, where we sample* $\mathbf{s} \in \Sigma^n$ *at random and set*

$$\mathbf{b}_i = f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)}).$$

For the lower bound in this section, *we assume that* $|\Sigma|$ *is a power of two.*

Formalizing the Proof System. Polynomial calculus refutations work as follows [CEI96]. We first initialize a system of polynomials $g_1(X) = 0, \ldots g_m(X) = 0$ in a set of variables X over some field. A new polynomial equation $g(X) = 0$ is appended to the system by either

1. Setting $g(X) = g'(X) + g''(X)$ for any polynomials $g'(X), g''(X)$ already in the system.
2. Setting $g(X) = X_i \cdot g'(X)$ for any variable X_i and any polynomial $g'(X)$ already in the system.

The goal of a refutation is to derive the statement $1 = 0$ via these operations; such a derivation is only possible if the original system was unsatisfiable, because the derivation rules cannot derive an unsatisfiable system from a satisfiable one. As in the lower bounds given by Applebaum and Lovett [AL16], we allow an adversary to have *nondeterministic power* when choosing the derivation steps she would like to perform. So the only measure of complexity of the proof will be its size, i.e. the total number of monomials in the proof after all polynomials are written in expanded form. This is natural in the sense that any algorithm executing a polynomial calculus refutation in the most natural manner will have to, at the very least, write out all of these monomials.[18]

For most applications, the initial system we'd like to refute already has a natural field structure. But for Problem 1 the functions in $\mathcal{F}$ are completely random. To handle this discrepancy, we fix the field $\mathbb{F}_2$ and then allow an adversary *unbounded time* to choose the best embedding from Σ, Γ into vectors over $\mathbb{F}_2$. We restrict ourselves to $\mathbb{F}_2$ because the polynomials representing each constraint $\mathbf{b}_i = f_i(\mathbf{s}_{N_{\mathbf{H}}(i,1)}, \ldots, \mathbf{s}_{N_{\mathbf{H}}(i,k)})$ will be of minimal degree when interpreted over $\mathbb{F}_2$ as opposed to a field of larger order, which is helpful for an adversary. Moreover, the random functions in $\mathcal{F}$ have no affinity for any particular field; indeed the only structured portion of Problem 1 for our cryptographic applications will be the matrix $\mathbf{H}$, which is of low complexity specifically with respect to $\mathbb{F}_2$. Now we formally define polynomial calculus refutations for Problem 1. Recall that we assume $|\Sigma|$ is a power of two.

Definition 6.12 (Polynomial Calculus Refutation for Problem 1)**.** *The refutation algorithm proceeds as follows. Let* X *be a set of* n *variables over* Σ*, interpreted as entries of the unknown secret* $\mathbf{s} \in \Sigma^n$*, and assume that the adversary has access to the entire tuple* $(\mathbf{H}, \mathcal{F}, \mathbf{b})$*.*

1. *The adversary non-deterministically finds the best (bijective) embeddings* $\phi_1, \ldots, \phi_n : \Sigma \to \mathbb{F}_2^{\log_2 |\Sigma|}$ *for each variable* X_j*. Let* $\mathbf{X}'$ *be a matrix of* $n \log_2 |\Sigma|$ *variables over* $\mathbb{F}_2$ *such that* $\mathbf{X}'_{j,1}, \ldots, \mathbf{X}'_{j,\log_2 |\Sigma|}$ *represents* X_j*.*

[18] A smarter algorithm could conceivably exploit nontrivial cancellations that occur without needing to fully distribute the polynomials. But it appears that the massive entropy coming from our random functions, in addition to the fact that each one is sampled independently at random, rules out such cancellations. In any case, counting the number of monomials in the polynomials' distributed form is standard for polynomial calculus lower bounds [AL16].

2. *The adversary initializes a polynomial system over $\mathbf{X}'$ by appending, for each $i \in [m]$, the constraint $f'_i(\mathbf{X}') = 0$, where f'_i is the unique polynomial over $\mathbb{F}_2$ such that*

   (a) *f'_i is supported on the variable set $\mathbf{X}'_{N_{\mathbf{H}}(i,1),1}, \ldots, \mathbf{X}'_{N_{\mathbf{H}}(i,1),\log_2|\Sigma|}, \mathbf{X}'_{N_{\mathbf{H}}(i,2),1}, \ldots, \mathbf{X}'_{N_{\mathbf{H}}(i,k),\log_2|\Sigma|}$.*

   (b) *For all $(\sigma_1, \ldots, \sigma_k) \in \Sigma^k$ such that $f_i(\sigma_1, \ldots, \sigma_k) = \mathbf{b}_i$, we have*

   $$f'_i(\phi_{N_{\mathbf{H}}(i,1)}(\sigma_1), \ldots, \phi_{N_{\mathbf{H}}(i,k)}(\sigma_k)) = 0.$$

   (c) *For all other inputs, f'_i equals 1.*

3. *The adversary (non-deterministically) appends a new polynomial $g(\mathbf{X}') = 0$ to the system, by either*

   (a) *Setting $g(\mathbf{X}') = g'(\mathbf{X}') + g''(\mathbf{X}')$ for any polynomials $g'(\mathbf{X}'), g''(\mathbf{X}')$ already in the system.*

   (b) *Setting $g(\mathbf{X}') = \mathbf{X}'_{j,\ell} \cdot g'(\mathbf{X}')$ for any variable $\mathbf{X}'_{j,\ell}$ and any polynomial $g'(\mathbf{X}')$ already in the system.*

The goal of the adversary is derive the equation $1 = 0$, and we measure the size of the proof by the number of nonzero monomials in the distributed form for each polynomial written at each step.

6.3.1 Proving the Lower Bound

As stated before, our techniques are similar to those used by [AL16] to prove polynomial calculus lower bounds for Goldreich's PRG. Informally speaking, we show that any polynomial calculus refutation for Problem 1 must have size at least $\exp\{n^{0.99}\}$.

Theorem 6.13. *Assume that $|\Sigma|$ is a power of two and that n is sufficiently large. With probability $1 - \exp\{-|\Sigma|^{\Omega(k)}\}$ over the choice of functions in $\mathcal{F}$, there does not exist a polynomial calculus refutation (in the sense of Definition 6.12) for a tuple $(\mathbf{H}, \mathcal{F}, \mathbf{b})$ drawn from the null distribution $\mathcal{Q}$ in Problem 1 containing less than $\exp\{n^{0.99}\}$ nonzero monomials.*

Before proving the theorem, we give a basic definition and import two key lemmas from [IPS99, AR01].

Definition 6.14 (Rational Degree)**.** *The* rational degree *of a polynomial $g : \mathbb{F}_2^w \to \mathbb{F}_2$ is the smallest integer d such that there exist degree-$\leq d$ polynomials $p, q : \mathbb{F}_2^w \to \mathbb{F}_2$, not both zero, such that $pg = q$.*

At an intuitive level, a polynomial g with low rational degree can be "converted" into a low degree polynomial for the purposes of a polynomial calculus refutation, even if the degree of g was originally very high. Rational degree will be of central importance in our lower bound, because demonstrating that the polynomials in our system have high rational degree will allow us to apply the following lemma.

Lemma 6.15 (Simplified version of Theorem 3.8 from [AR01])**.** *Suppose we have a polynomial system $g_1(X) = 0, \ldots, g_m(X) = 0$ over a set X of variables in $\mathbb{F}_2$. Assume that*

1. *Every polynomial depends on at most k' variables.*
2. *For all subsets $\mathcal{G}$ of at least 1 polynomial and at most t polynomials, $\mathcal{G}$ depends on at least $(1-o(1))tk'$ distinct variables.*
3. *Every polynomial has rational degree $\Omega(k')$.*

Then any polynomial calculus refutation of the system $g_1(X) = 0, \ldots, g_m(X) = 0$ has at least one monomial of degree $\Omega(tk')$.

Remark 6.16. The conditions we impose on the polynomial system in this lemma are equivalent to those in Theorem 3.8 from [AR01], but phrased slightly differently. As stated by Alekhnovich and Razborov [AR01], the lemma actually requires the polynomial system to have strong *boundary expansion*, but our notion of expansion in Item 2 immediately implies the required boundary expansion. Additionally, [AR01] require the polynomials to satisfy a condition they call *immunity*, which is implied by our notion of rational degree. We adopt the rational degree perspective because it aligns with the techniques used by [AL16].

The lemma below allows us to say that the existence of a large monomial in the proof implies that the proof itself is very long.

Lemma 6.17 ([IPS99] Theorem 6.2)**.** *If G is a set of polynomials in n variables, each of degree at most d, and G has a polynomial calculus refutation with M nonzero monomials, then G has a polynomial calculus refutation of degree at most* $\max(d, 2\left\lceil\sqrt{n \log M}\right\rceil + 1)$.

Now we begin working towards a proof of Theorem 6.13. Below we show that, even though an adversary can choose arbitrary embeddings from Σ to $\mathbb{F}_2^{\log_2|\Sigma|}$ for each variable representing the secret $\mathbf{s}$ in Problem 1, all of the constraints will have high rational degree with high probability. The lemma just formalizes the intuition that the random functions in $\mathcal{F}$ have far too much entropy to be summarized by a low degree polynomial.

Lemma 6.18. *Let n be a parameter, and let $k = (\log n)^{\Theta(1)}$ and $m \leq n^{o(k)}$. Choose any alphabets Σ, Γ satisfying $|\Sigma| = (nm)^{\log^{\Theta(1)}(nm)}$ and $|\Gamma| \leq |\Sigma|^{3k/4}$. Sample m random functions $f_i : \Sigma^k \to \Gamma$. Then with probability $1 - \exp\left\{-|\Sigma|^{\Omega(k)}\right\}$, there does not exist an index $i \in [m]$, a value $\mathbf{b}_i \in \Gamma$, and a set of (bijective) embeddings $\phi_1, \ldots, \phi_k : \Sigma \to \mathbb{F}_2^{\log_2|\Sigma|}$ such that f_i' has rational degree less than $\frac{k \log_2|\Sigma|}{100}$. Here f_i' is the unique polynomial over $\mathbb{F}_2$ such that*

1. *f_i' is supported on the variable set $\mathbf{X}'_{N_{\mathbf{H}}(i,1),1}, \ldots, \mathbf{X}'_{N_{\mathbf{H}}(i,1),\log_2|\Sigma|}, \mathbf{X}'_{N_{\mathbf{H}}(i,2),1}, \ldots, \mathbf{X}'_{N_{\mathbf{H}}(i,k),\log_2|\Sigma|}$.*
2. *For all $(\sigma_1, \ldots, \sigma_k) \in \Sigma^k$ such that $f_i(\sigma_1, \ldots, \sigma_k) = \mathbf{b}_i$, we have*
$$f_i'(\phi_{N_{\mathbf{H}}(i,1)}(\sigma_1), \ldots, \phi_{N_{\mathbf{H}}(i,k)}(\sigma_k)) = 0.$$
3. *For all other inputs, f_i' equals 1.*

Proof. First we give a sharp tail bound on the probability that, for a single choice of index i, value $\mathbf{b}_i$, embeddings $\phi_1, \ldots, \phi_k$, and low degree polynomials p, q, a random function f_i will satisfy $pf_i = q$. Then we union bound over all choices of $i, \mathbf{b}_i, \phi_1, \ldots, \phi_k, p, q$ to complete the proof.

Claim 6.19. *Fix an index $i \in [m]$, a value $\mathbf{b}_i \in \Gamma$, a set of (bijective) embeddings $\phi_1, \ldots, \phi_k : \Sigma \to \mathbb{F}_2^{\log_2|\Sigma|}$, and two degree-$\leq \frac{k \log_2|\Sigma|}{100}$ polynomials $p, q : \mathbb{F}_2^{k \log_2|\Sigma|} \to \mathbb{F}_2$, at least one of which is nonzero. Then sample a random function $f_i : \Sigma^k \to \Gamma$, and define f_i' as in the statement of the lemma. With probability $1 - \exp\left\{-\Omega(|\Sigma|^{\frac{24}{100}k})\right\}$, we have $p \cdot f_i' \neq q$.*

Proof. First consider the case that p is identically zero. Then $p \cdot f_i'$ is identically zero, in which case $p \cdot f_i' = q$ if and only if q is identically zero. This contradicts the assumption in the claim.

Now assume that p is nonzero and q is any (potentially identically zero) polynomial, and let $\mathcal{S}$ be the set of points for which p equals one. For every point in $\mathcal{S}$, the evaluation of $p \cdot f_i'$ is equal to the evaluation of f_i', so a necessary condition to have $p \cdot f_i' = q$ is for f_i' to equal q on all points in $\mathcal{S}$.

Because of the degree bound on p, we have that the truth table of p is a member of the Reed-Muller code $\mathrm{RM}(k \log_2 |\Sigma|, \frac{k \log_2 |\Sigma|}{100})$. By Lemma 2.9, the minimum distance of $\mathrm{RM}(k \log_2 |\Sigma|, \frac{k \log_2 |\Sigma|}{100})$ is

$$2^{k \log_2 |\Sigma| - \frac{k \log_2 |\Sigma|}{100}} = 2^{\frac{99}{100} k \log_2 |\Sigma|},$$

which implies that $|\mathcal{S}| \geq 2^{\frac{99}{100} k \log_2 |\Sigma|}$. Since $\mathbf{b}_i$, the (bijective) embeddings $\phi_1, \ldots, \phi_k$, and the polynomial p were all fixed before sampling f'_i, each point in $\mathcal{S}$ has the evaluation of f'_i equal to zero independently with probability $1 - 1/|\Gamma|$. Since q was also fixed before sampling f_i, the probability that f'_i and q have equal evaluations on all points in $\mathcal{S}$ is at most $(1 - 1/|\Gamma|)^{|\mathcal{S}|}$. Using that $|\Gamma| \leq |\Sigma|^{3k/4}$, the assumption that Σ is sufficiently large, and the fact that $(1 - 1/x)^x < 1/2$ for sufficiently large x, the probability that f_i and q have equal evaluations for all points in $\mathcal{S}$ is upper bounded as

$$\begin{aligned}
(1 - 1/|\Gamma|)^{2^{\frac{99}{100} k \log_2 |\Sigma|}} &\leq (1 - |\Sigma|^{-3k/4})^{2^{\frac{99}{100} k \log_2 |\Sigma|}} \\
&< (1/2)^{2^{\frac{24}{100} k \log_2 |\Sigma|}} \\
&= 2^{-|\Sigma|^{\frac{24}{100} k}}
\end{aligned}$$

□

All that's left is to union bound over all choices of $i \in [m]$, $\mathbf{b}_i \in \Gamma$, bijective embeddings $\phi_1, \ldots, \phi_k : \Sigma \to \mathbb{F}_2^{\log_2 |\Sigma|}$, and degree-$\leq \frac{k \log_2 |\Sigma|}{100}$ polynomials $p, q : \mathbb{F}_2^{k \log_2 |\Sigma|} \to \mathbb{F}_2$ (we also count the extraneous case that both p and q are zero). Using the standard approximation that $\binom{x}{y} < (3x/y)^y$, there are

1. m choices for the index i.
2. $(|\Sigma|!)^k \leq |\Sigma|^{k|\Sigma|}$ choices for the embeddings $\phi_1, \ldots, \phi_k$
3. $2^{\binom{k \log_2 |\Sigma|}{\frac{k \log_2 |\Sigma|}{100}}} \leq 2^{300 \frac{k \log_2 |\Sigma|}{100}} < 2^{|\Sigma|^{\frac{k}{10}}}$ choices for the polynomial p.
4. $2^{\binom{k \log_2 |\Sigma|}{\frac{k \log_2 |\Sigma|}{100}}} \leq 2^{300 \frac{k \log_2 |\Sigma|}{100}} < 2^{|\Sigma|^{\frac{k}{10}}}$ choices for the polynomial q.

Using that $k = (\log n)^{\Theta(1)} = (\log m)^{\Theta(1)}$ and $|\Sigma| = m^{\omega(1)}$, and assuming $|\Sigma|$ is sufficiently large, there are

$$m \cdot |\Sigma|^{k|\Sigma|} \cdot 2^{|\Sigma|^{\frac{k}{10}}} \cdot 2^{|\Sigma|^{\frac{k}{10}}} < 2^{|\Sigma|^{k/9}}$$

total choices. So by Claim 6.19 and the definition of rational degree, the probability that there exists an index $i \in [m]$, a value $\mathbf{b}_i \in \Gamma$, and a set of (bijective) embeddings $\phi_1, \ldots, \phi_k : \Sigma \to \mathbb{F}_2^{\log_2 |\Sigma|}$ such that f'_i has rational degree less than $\frac{k \log_2 |\Sigma|}{100}$ is at most

$$2^{|\Sigma|^{k/9}} \cdot 2^{-\Omega(|\Sigma|^{\frac{24}{100} k})} = \exp\left\{-|\Sigma|^{\Omega(k)}\right\}.$$

□

Now we are ready to prove the lower bound on polynomial calculus refutations.

Proof of Theorem 6.13. Condition on the event in Lemma 6.18 occurring for the set of functions $\mathcal{F}$, which occurs with probability $1 - \exp\left\{-|\Sigma|^{\Omega(k)}\right\}$. In this case, every function f_i' derived from each $(f_i, \mathbf{b}_i)$ pair will have rational degree $\Omega(k \log_2 |\Sigma|)$, regardless of what the $\mathbf{b}_i$ values are and regardless of what embeddings $\phi_1, \ldots, \phi_n : \Sigma \to \mathbb{F}_2^{\log_2 |\Sigma|}$ the adversary picks.

Now examine the polynomial system $f_1' = 0, \ldots, f_m' = 0$, which is the system (over $\mathbb{F}_2$) for which the adversary wishes to find a short refutation. Because $\log_2 |\Sigma| = \log^{\Theta(1)} n$, we know that there will be $n \log_2 |\Sigma| = n \log^{\Theta(1)} n$ variables. We know by assumption on the matrix $\mathbf{H}$ and by Lemma 6.18 that the polynomial system satisfies the preconditions of Lemma 6.15 for an expansion cutoff $t = n^{1-o(1)}$, so any polynomial calculus refutation must have at least one nonzero monomial of degree at least $n^{1-o(1)}$.

Now suppose for contradiction that there exists a polynomial calculus refutation for this system having less than $\exp\left\{n^{0.99}\right\}$ nonzero monomials. Observe that, because we are working over $\mathbb{F}_2$, every polynomial f_i' can have degree at most the number of variables it depends on, which is $\log^{\Theta(1)} n$. By Lemma 6.17, all of this taken together with the assumption that the problem size is sufficiently large implies that there is a polynomial calculus refutation of degree at most

$$\max\left(\log^{\Theta(1)} n, 2\left\lceil\sqrt{n \log^{\Theta(1)} \cdot n^{0.99}}\right\rceil + 1\right) = n^{0.995+o(1)} < n^{1-o(1)},$$

which is not possible. □

7 References

[ABW10] Benny Applebaum, Boaz Barak, and Avi Wigderson. Public-key cryptography from different assumptions. In *Proceedings of the forty-second ACM symposium on Theory of computing*, pages 171–180, 2010. [1, 2, 3, 6, 11]

[AL16] Benny Applebaum and Shachar Lovett. Algebraic attacks against random local functions and their countermeasures. In *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing*, pages 1087–1100, 2016. [1, 5, 6, 41, 42, 43, 44]

[AL17] Vedat Levi Alev and Lap Chi Lau. Approximating unique games using low diameter graph decomposition. *arXiv preprint arXiv:1702.06969*, 2017. [2]

[Ale03] Michael Alekhnovich. More on average case vs approximation complexity. In *44th Annual IEEE Symposium on Foundations of Computer Science, 2003. Proceedings.*, pages 298–307. IEEE, 2003. [1]

[App12] Benny Applebaum. Pseudorandom generators with long stretch and low locality from random local one-way functions. In *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*, pages 805–816, 2012. [10]

[AR01] Michael Alekhnovich and Alexander A Razborov. Lower bounds for polynomial calculus: Non-binomial case. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pages 190–199. IEEE, 2001. [43, 44]

[AR16] Benny Applebaum and Pavel Raykov. Fast pseudorandom functions based on expander graphs. In *Theory of Cryptography Conference*, pages 27–56. Springer, 2016. [4]

[ASW15] Emmanuel Abbe, Amir Shpilka, and Avi Wigderson. Reed-muller codes for random erasures and errors. In *Proceedings of the forty-seventh annual ACM symposium on Theory of Computing*, pages 297–306, 2015. [14]

[ASY20] Emmanuel Abbe, Amir Shpilka, and Min Ye. Reed–muller codes: Theory and algorithms. *IEEE Transactions on Information Theory*, 67(6):3251–3277, 2020. [14]

[BBH+20] Matthew Brennan, Guy Bresler, Samuel B Hopkins, Jerry Li, and Tselil Schramm. Statistical query algorithms and low-degree tests are almost equivalent. *arXiv preprint arXiv:2009.06107*, 2020. [5]

[BBK+21] Afonso S Bandeira, Jess Banks, Dmitriy Kunisky, Christopher Moore, and Alex Wein. Spectral planting and the hardness of refuting cuts, colorability, and communities in random graphs. In *Conference on Learning Theory*, pages 410–473. PMLR, 2021. [5]

[BCG+12] Aditya Bhaskara, Moses Charikar, Venkatesan Guruswami, Aravindan Vijayaraghavan, and Yuan Zhou. Polynomial integrality gaps for strong sdp relaxations of densest k-subgraph. In *Proceedings of the twenty-third annual ACM-SIAM symposium on Discrete Algorithms*, pages 388–405. SIAM, 2012. [2]

[BHJK25] Rares-Darius Buhai, Jun-Ting Hsieh, Aayush Jain, and Pravesh K Kothari. The quasi-polynomial low-degree conjecture is false. *arXiv preprint arXiv:2505.17360*, 2025. [3]

[BJRZ24] Andrej Bogdanov, Chris Jones, Alon Rosen, and Ilias Zadik. Low-degree security of the planted random subgraph problem. In *Theory of Cryptography Conference*, pages 255–275. Springer, 2024. [5]

[BKR23] Andrej Bogdanov, Pravesh K Kothari, and Alon Rosen. Public-key encryption, local pseudo-random generators, and the low-degree method. In *Theory of Cryptography Conference*, pages 268–285. Springer, 2023. [5, 11]

[BKW03] Avrim Blum, Adam Kalai, and Hal Wasserman. Noise-tolerant learning, the parity problem, and the statistical query model. *Journal of the ACM (JACM)*, 50(4):506–519, 2003. [6]

[BKW19] Afonso S Bandeira, Dmitriy Kunisky, and Alexander S Wein. Computational hardness of certifying bounds on constrained pca problems. *arXiv preprint arXiv:1902.07324*, 2019. [5]

[Bra08] Mark Braverman. Polylogarithmic independence fools ac 0 circuits. *Journal of the ACM (JACM)*, 57(5):1–10, 2008. [6]

[BSI99] Eli Ben-Sasson and Russell Impagliazzo. Random cnfs are hard for the polynomial calculus. In *40th Annual Symposium on Foundations of Computer Science (Cat. No. 99CB37039)*, pages 415–421. IEEE, 1999. [1, 5]

[CEI96] Matthew Clegg, Jeffery Edmonds, and Russell Impagliazzo. Using the groebner basis algorithm to find proofs of unsatisfiability. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 174–183, 1996. [5, 42]

[CIKK16] Marco L Carmosino, Russell Impagliazzo, Valentine Kabanets, and Antonina Kolokolova. Learning algorithms from natural proofs. In *31st Conference on Computational Complexity (CCC 2016)*, pages 10–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2016. [4, 29]

[CMMV17] Eden Chlamtáč, Pasin Manurangsi, Dana Moshkovitz, and Aravindan Vijayaraghavan. Approximation algorithms for label cover and the log-density threshold. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 900–919. SIAM, 2017. [1, 2]

[CSZ24] Xue Chen, Wenxuan Shu, and Zhaienhe Zhou. Algorithms for sparse lpn and lspn against low-noise. *arXiv preprint arXiv:2407.19215*, 2024. [6]

[DDL23] Jian Ding, Hang Du, and Zhangsong Li. Low-degree hardness of detection for correlated erd\h {o} sr\'enyi graphs. *arXiv preprint arXiv:2311.15931*, 2023. [5]

[DH76] WHITFIELD DIFFIE and MARTIN E HELLMAN. New directions in cryptography. *IEEE TRANSACTIONS ON INFORMATION THEORY*, 22(6), 1976. [1]

[DMW25] Abhishek Dhawan, Cheng Mao, and Alexander S Wein. Detection of dense subhypergraphs by low-degree polynomials. *Random Structures & Algorithms*, 66(1):e21279, 2025. [1, 37]

[EH25] Dor Elimelech and Wasim Huleihel. Detecting arbitrary planted subgraphs in random graphs. *arXiv preprint arXiv:2503.19069*, 2025. [4]

[FGKP06] Vitaly Feldman, Parikshit Gopalan, Subhash Khot, and Ashok Kumar Ponnuswami. New results for learning noisy parities and halfspaces. In *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS'06)*, pages 563–574. IEEE, 2006. [6]

[FGR+17] Vitaly Feldman, Elena Grigorescu, Lev Reyzin, Santosh S Vempala, and Ying Xiao. Statistical algorithms and a lower bound for detecting planted cliques. *Journal of the ACM (JACM)*, 64(2):1–37, 2017. [5]

[FPV15] Vitaly Feldman, Will Perkins, and Santosh Vempala. On the complexity of random satisfiability problems with planted solutions. In *Proceedings of the forty-seventh annual ACM symposium on Theory of Computing*, pages 77–86, 2015. [1, 7]

[GGH+16] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. *SIAM Journal on Computing*, 45(3):882–929, 2016. [3]

[GHJS25] Riddhi Ghosal, Isaac M Hair, Aayush Jain, and Amit Sahai. Using the planted clique conjecture for cryptography: Public-key encryption from planted clique and noisy k-lin over expanders. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, pages 1921–1932, 2025. [4, 11]

[GM84] Shafi Goldwasser and Silvio Micali. Probabilistic encryption. In *Journal of Computer and System Sciences*, volume 28(2), pages 270–299. 1984. [1, 2, 13]

[Gol00] Oded Goldreich. Candidate one-way functions based on expander graphs. 2000. [1, 4]

[Gri01] Dima Grigoriev. Linear lower bound on degrees of positivstellensatz calculus proofs for the parity. *Theoretical Computer Science*, 259(1-2):613–622, 2001. [1, 6]

[GS11] Venkatesan Guruswami and Ali Kemal Sinop. Lasserre hierarchy, higher eigenvalues, and approximation schemes for graph partitioning and quadratic integer programming with psd objectives. In *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science*, pages 482–491. IEEE, 2011. [2]

[GTK15] Shafi Goldwasser and Yael Tauman Kalai. Cryptographic assumptions: A position paper. In *Theory of Cryptography Conference*, pages 505–522. Springer, 2015. [2]

[HKP+17] Samuel B Hopkins, Pravesh K Kothari, Aaron Potechin, Prasad Raghavendra, Tselil Schramm, and David Steurer. The power of sum-of-squares for detecting hidden structures. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 720–731. IEEE, 2017. [5]

[Hop18] Samuel Hopkins. *Statistical inference and the sum of squares method*. Cornell University, 2018. [3, 37, 38]

[HR05] Thomas Holenstein and Renato Renner. One-way secret-key agreement and applications to circuit polarization and immunization of public-key encryption. In *Annual International Cryptology Conference*, pages 478–493. Springer, 2005. [8, 12, 13]

[IPS99] Russell Impagliazzo, Pavel Pudlák, and Jiri Sgall. Lower bounds for the polynomial calculus and the gröbner basis algorithm. *Computational Complexity*, 8(2):127–144, 1999. [1, 5, 43, 44]

[Jan04] Svante Janson. Large deviations for sums of partly dependent random variables. *Random Structures & Algorithms*, 24(3):234–248, 2004. [33]

[JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In *STOC*, 2021. [3]

[KMOW17] Pravesh K Kothari, Ryuhei Mori, Ryan O'Donnell, and David Witmer. Sum of squares lower bounds for refuting any csp. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*, pages 132–145, 2017. [1, 6]

[Kun21] Dmitriy Kunisky. Hypothesis testing with low-degree polynomials in the morris class of exponential families. In *Conference on Learning Theory*, pages 2822–2848. PMLR, 2021. [5]

[KVWX23] Pravesh Kothari, Santosh S Vempala, Alexander S Wein, and Jeff Xu. Is planted coloring easier than planted clique? In *The Thirty Sixth Annual Conference on Learning Theory*, pages 5343–5372. PMLR, 2023. [5]

[KWB19] Dmitriy Kunisky, Alexander S Wein, and Afonso S Bandeira. Notes on computational hardness of hypothesis testing: Predictions using the low-degree likelihood ratio. In *ISAAC Congress (International Society for Analysis, its Applications and Computation)*, pages 1–50. Springer, 2019. [5, 37, 38]

[KZ17] Andrei Krokhin and Stanislav Zivny. *The constraint satisfaction problem: Complexity and approximability*. Schloss Dagstuhl, 2017. [1]

[LG24] Yuetian Luo and Chao Gao. Computational lower bounds for graphon estimation via low-degree polynomials. *The Annals of Statistics*, 52(5):2318–2348, 2024. [5]

[Lyu05] Vadim Lyubashevsky. The parity problem in the presence of noise, decoding random linear codes, and the subset sum problem. In *International Workshop on Approximation Algorithms for Combinatorial Optimization*, pages 378–389. Springer, 2005. [6]

[Man26] Breaking the alekhnovich barrier via a combinatorial list recovery conjecture. *Unpublished Manuscript*, 2026. [1, 2]

[McE78] Robert J McEliece. A public-key cryptosystem based on algebraic. *Coding Thv*, 4244(1978):114–116, 1978. [1, 11]

[MN24] Mladen Mikša and Jakob Nordström. A generalized method for proving polynomial calculus degree lower bounds. *Journal of the ACM*, 71(6):1–43, 2024. [1, 5]

[MSZ16] Eric Miles, Amit Sahai, and Mark Zhandry. Annihilation attacks for multilinear maps: Cryptanalysis of indistinguishability obfuscation over ggh13. In *Annual international cryptology conference*, pages 629–658. Springer, 2016. [3]

[Mul54] David E Muller. Application of boolean algebra to switching circuit design and to error detection. *Transactions of the IRE professional group on electronic computers*, (3):6–12, 1954. [13]

[MVW24] Jay Mardia, Kabir Aladin Verchand, and Alexander S Wein. Low-degree phase transitions for detecting a planted clique in sublinear time. In *The Thirty Seventh Annual Conference on Learning Theory*, pages 3798–3822. PMLR, 2024. [37]

[MW16] Ryuhei Mori and David Witmer. Lower bounds for csp refutation by sdp hierarchies. *arXiv preprint arXiv:1610.03029*, 2016. [1]

[MW25] Andrea Montanari and Alexander S Wein. Equivalence of approximate message passing and low-degree polynomials in rank-one matrix estimation. *Probability Theory and Related Fields*, 191(1):181–233, 2025. [5]

[Nor14] Jakob Nordström. A (biased) proof complexity survey for sat practitioners. In *International Conference on Theory and Applications of Satisfiability Testing*, pages 1–6. Springer, 2014. 1, 5

[NW94] Noam Nisan and Avi Wigderson. Hardness vs randomness. *Journal of computer and System Sciences*, 49(2):149–167, 1994. 10

[OST18] I Oliveira, Rahul Santhanam, and Roei Tell. Expander-based cryptography meets natural proofs. *Innovations in Theoretical Computer Science (ITCS)*, 124, 2018. 1, 2, 4, 10, 11, 29, 32

[Rab79] Michael O Rabin. Digitalized signatures and public-key functions as intractable as factorization. Technical report, 1979. 1

[Raz87] Alexander A Razborov. Lower bounds on the size of bounded depth circuits over a complete basis with logical addition. *Mathematical Notes of the Academy of Sciences of the USSR*, 41(4):333–338, 1987. 4, 29

[Ree53] Irving S Reed. A class of multiple-error-correcting codes and the decoding scheme. Technical report, 1953. 13

[Reg09] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009. 1

[RR94] Alexander A Razborov and Steven Rudich. Natural proofs. In *Proceedings of the twenty-sixth annual ACM symposium on Theory of computing*, pages 204–213, 1994. 4, 29

[RSA78] Ronald L Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. 1

[Sah26] Amit Sahai. A note on adversary running times. *Cryptology ePrint Archive*, 2026. 2

[Sch08] Grant Schoenebeck. Linear level lasserre lower bounds for certain k-csps. In *2008 49th Annual IEEE Symposium on Foundations of Computer Science*, pages 593–602. IEEE, 2008. 1, 6

[Sho99] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999. 1

[Smo87] Roman Smolensky. Algebraic methods in the theory of lower bounds for boolean circuit complexity. In *Proceedings of the nineteenth annual ACM symposium on Theory of computing*, pages 77–82, 1987. 4, 29

[SSV16] Ramprasad Saptharishi, Amir Shpilka, and Ben Lee Volk. Efficiently decoding reed-muller codes from random errors. In *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing*, pages 227–235, 2016. 10, 14, 26, 27

[Ste10] David Steurer. *On the complexity of unique games and graph expansion*. Princeton University, 2010. 2

[Val12] Gregory Valiant. Finding correlations in subquadratic time, with applications to learning parities and juntas. In *2012 IEEE 53rd Annual Symposium on Foundations of Computer Science*, pages 11–20. IEEE, 2012. 6

[WEAM19] Alexander Wein, Ahmed El Alaoui, and Cristopher Moore. The kikuchi hierarchy and tensor pca. *Journal of the ACM*, 2019. 1, 7

[Wei22] Alexander S Wein. Optimal low-degree hardness of maximum independent set. *Mathematical Statistics and Learning*, 4(3):221–251, 2022. [5]

[Wei23] Alexander S Wein. Average-case complexity of tensor decomposition for low-degree polynomials. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 1685–1698, 2023. [5]

[YZ16] Yu Yu and Jiang Zhang. Cryptography with auxiliary input and trapdoor from constant-noise lpn. In *Annual International Cryptology Conference*, pages 214–243. Springer, 2016. [2]

[YZZ24] Xifan Yu, Ilias Zadik, and Peiyuan Zhang. Counting stars is constant-degree optimal for detecting any planted subgraph. In *The Thirty Seventh Annual Conference on Learning Theory*, pages 5163–5165. PMLR, 2024. [5]

[Zhu20] Dmitriy Zhuk. A proof of the csp dichotomy conjecture. *Journal of the ACM (JACM)*, 67(5):1–78, 2020. [1]